



HY340 : ΓΛΩΣΣΕΣ ΚΑΙ ΜΕΤΑΦΡΑΣΤΕΣ

ΠΑΝΕΠΙΣΤΗΜΙΟ ΚΡΗΤΗΣ,
ΣΧΟΛΗ ΘΕΤΙΚΩΝ ΕΠΙΣΤΗΜΩΝ,
ΤΜΗΜΑ ΕΠΙΣΤΗΜΗΣ ΥΠΟΛΟΓΙΣΤΩΝ

```
VAR i:Integer;  
FUNCTION(Symbol) replicate  
    x = (function(x,y){return x+y;});  
    class DelFunctor: public std::unary_function<
```

ΔΙΔΑΣΚΩΝ

Αντώνιος Σαββίδης



HY340 : ΓΛΩΣΣΕΣ ΚΑΙ ΜΕΤΑΦΡΑΣΤΕΣ

Διάλεξη 6η ΣΥΝΤΑΚΤΙΚΗ ΑΝΑΛΥΣΗ - III

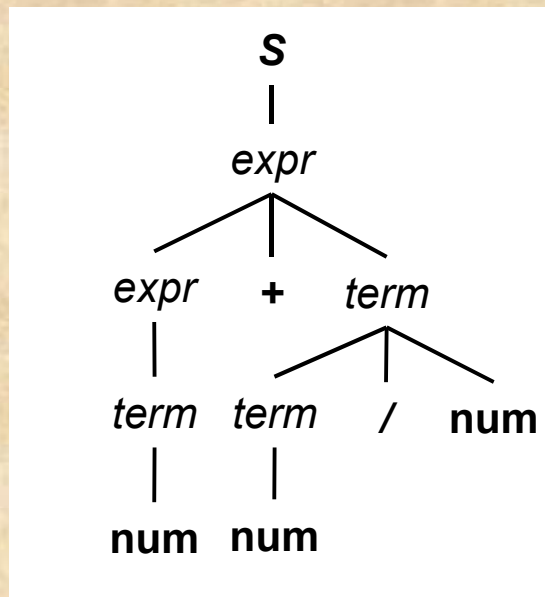


Περιεχόμενα

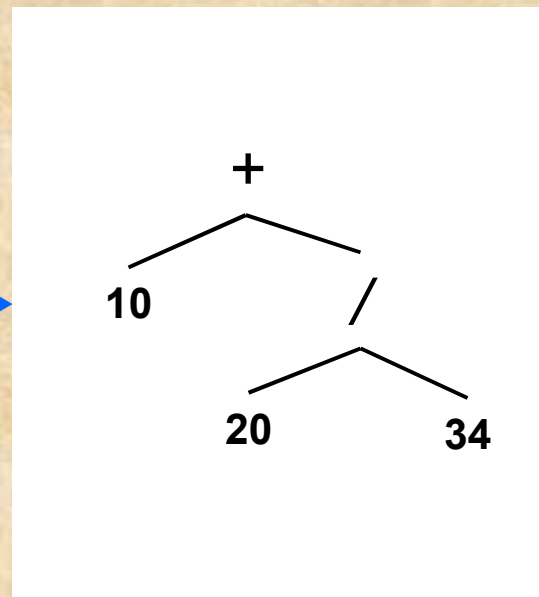
- *Αφηρημένα συντακτικά δέντρα*
- Ανοδική συντακτική ανάλυση
- Ασυμφωνίες στην ανοδική ανάλυση
- Αναλυτές LR

Αφηρημένα συντακτικά δέντρα (1/4)

- Πρόκειται για μία απλοποιημένη μορφή του δέντρου συντακτικής ανάλυσης κατάλληλη για την αναπαράσταση δομών της γλώσσας
- Αρκετές φορές χρησιμοποιούνται και αντίστοιχοι τύποι για την εσωτερική αποθήκευση της συντακτικής δομής



Δέντρο συντακτικής ανάλυσης



Αφηρημένο συντακτικό δέντρο

• Στα αφηρημένα συντακτικά δέντρα είναι πολύ συνηθισμένη η εξάλειψη διαδοχικών μονών παραγωγών, π.χ. $expr \Rightarrow term \Rightarrow num$

• Συνήθως το συντακτικό δέντρο δεν κατασκευάζεται αλλά απλώς «ακολουθείται» από τις κινήσεις του αναλυτή, ενώ το αφηρημένο δέντρο δημιουργείται από τον αναλυτή κατά τη διάρκεια της συντακτικής ανάλυσης



Αφηρημένα συντακτικά δέντρα (2/4)

- Η κατασκευή ενός αφηρημένου δέντρου γίνεται σε έναν καθοδικό αναλυτή εύκολα
 - αλλά από κάτω προς τα πάνω
 - όταν έχουμε επιτυχή επιστροφή από αναδρομικές κλήσεις
- Αυτό είναι ιδιαίτερα εύκολο με έναν RDP καθώς το δέντρο κλήσεων των αναδρομικών συναρτήσεων είναι ακριβώς το ίδιο με το δέντρο συντακτικής ανάλυσης
 - αφού οι κλήσεις εξομοιώνουν τις παραγωγές
 - όμως οι μεταβολές στο δέντρο λόγω μετασχηματισμών (left recursion elimination, left factoring) οδηγούν σε όχι άμεσα προφανές χτίσιμο του δέντρου, αλλά υλοποιήσιμο εύκολα
- Αρκεί ταυτόχρονα με την ανάλυση να δημιουργούμε τους κόμβους του δέντρου
 - ή να τους καταστρέψουμε εάν έχουμε μερική επιτυχία της ανάλυσης



Αφηρημένα συντακτικά δέντρα (3/4)

```
enum node_t { expr_c, while_c, ifelse_c };
struct node {
    node_t type;
    void* data;
};

enum expr_t { add_c, sub_c, mul_c, div_c, num_c, id_c };
struct expr_node {
    expr_t type;
    union {
        double numVal;
        char* varVal;
    } data;
    expr_node* left;
    expr_node* right;
};

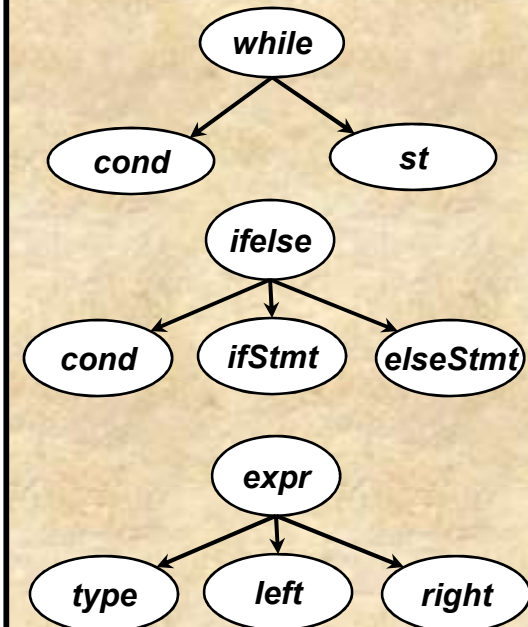
struct while_node {
    expr_node* cond;
    node* st;
};

node* make_while (expr_node* cond, node* stmt);

struct ifelse_node {
    expr_node* cond;
    node* ifStmt;
    node* elseStmt;
};

node* make_ifelse (expr_node* cond, node* ifStmt, node* elseStmt);
```

while → **WHILE** (*cond*) *stmt*
cond → *expr*
ifelse → **IF** (*cond*) *stmt* *else*
else → **ELSE** *stmt* | ϵ
stmt → *while* | *ifelse* | ;





Αφηρημένα συντακτικά δέντρα (4/4)

Δηλ. το αφηρημένο συντακτικό δέντρο (AST) δεν είναι τελικά και τόσο αφηρημένο, καθώς αποτυπώνει την εσωτερική δομή αποθήκευσης στο πρόγραμμά μας του πραγματικού συντακτικού δέντρου, από το οποίο όμως έχουμε αφαιρέσει κάθε μη σημασιολογικά απαραίτητη πληροφορία. Το AST δεν είναι απαραίτητο να έχει ισομορφική δομή με το δέντρο συντακτικής ανάλυσης. Έτσι, ενώ το parse tree διαγράφει τις ακριβείς «κινήσεις» συντακτικής ανάλυσης, αλλά δεν αποθηκεύεται σχεδόν ποτέ, το AST είναι το πραγματικό παράγωγο της ανάλυσης πάνω στο οποίο και θα «δουλέψει» μετέπειτα η μετάφραση.

```
node* while_f (void) {
    if (lookAhead!= WHILE_TOKEN)
        return (node*) 0;
    if (!match(OPENPAR_TOKEN)) {
        error(OPENPAR_TOKEN);
        return (node*) 0;
    }
    expr_node* cond = expr_f();
    if (!cond)
        return (node*) 0;
    if (!match(CLOSEPAR_TOKEN)) {
        error(CLOSEPAR_TOKEN);
        free(cond);
        return (node*) 0;
    }
    node* stmt = stmt_f();
    if (!stmt) {
        free(cond);
        return (node*) 0;
    }
    return make_while(cond, stmt);
}
```

```
node* ifelse_f (void) {
    if (lookAhead!= IF_TOKEN)
        return (node*) 0;
    if (!match(OPENPAR_TOKEN)) {
        error(OPENPAR_TOKEN);
        return (node*) 0;
    }
    expr_node* cond = expr_f();
    if (!cond)
        return (node*) 0;
    if (!match(CLOSEPAR_TOKEN)) {
        error(CLOSEPAR_TOKEN);
        free(cond);
        return (node*) 0;
    }
    node* ifStmt = stmt_f();
    if (!ifStmt) {
        free(cond);
        return (node*) 0;
    }

    node* elseStmt = (node*) 0;
    if (lookAhead==ELSE_TOKEN && match(ELSE_TOKEN))
        elseStmt = stmt_f();
    return make_ifelse(cond, ifStmt, elseStmt);
}
```

Με τον τρόπο αυτό επιλύουμε αλγοριθμικά το πρόβλημα της διφορούμενης γραμματικής if..else, γιατί δεν υπάρχει περίπτωση else μετά από if να μην ενωθούν στο ίδιο συντακτικό υπόδεντρο.



Περιεχόμενα

- Αφηρημένα συντακτικά δέντρα
- *Ανοδική συντακτική ανάλυση*
- Ασυμφωνίες στην ανοδική ανάλυση
- Αναλυτές LR



Ανοδική συντακτική ανάλυση (1/30)

- Κατά την ανοδική συντακτική ανάλυση ο προσδιορισμός του δέντρου συντακτικής ανάλυσης γίνεται από τα φύλλα προς τη ρίζα
- Οι ανοδικοί αναλυτές κοιτούν την είσοδο από αριστερά προς δεξιά και προσπαθούν να προσδιορίσουν την παραγωγή που οδήγησε στο παρόν τερματικό σύμβολο
- Μαντεύοντας με κάποιο τρόπο και επιλέγοντας, χτίζουν το δέντρο διαδοχικά προς τα πάνω, συνδέοντας εν ανάγκη υπόδεντρα, έως ότου φτάσουν στο αρχικό σύμβολο και έχει καταναλωθεί ή είσοδος

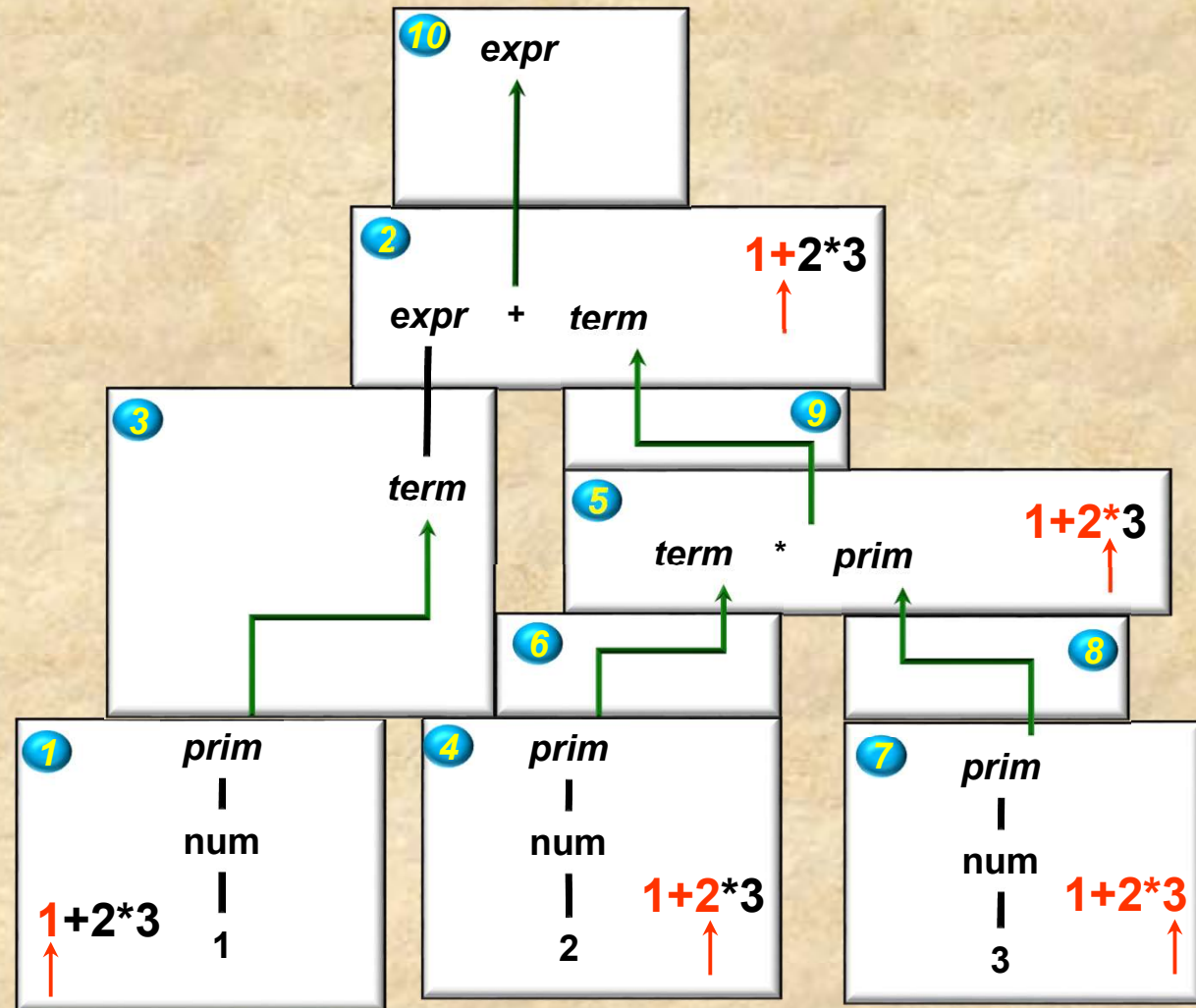


Ανοδική συντακτική ανάλυση (2/30)

S	$\rightarrow expr$
$expr$	$\rightarrow expr \text{ addsub } term$
$expr$	$\rightarrow term$
$term$	$\rightarrow term \text{ muldiv } prim$
$term$	$\rightarrow prim$
$prim$	$\rightarrow num$
$prim$	$\rightarrow (expr)$

Προσδιορισμός του parse tree
μίας λέξης εφαρμόζοντας
τεχνική ανοδικής ανάλυσης
και δημιουργίας του δέντρου
από υπόδεντρα

Φτιάξε το δέντρο ώστε να
έχεις χρησιμοποιήσει όλη την
είσοδο και στη ρίζα να είναι
το αρχικό σύμβολο της
γραμματικής





Ανοδική συντακτική ανάλυση (3/30)

- Θα δούμε μία πιο τυποποιημένη έκφραση αυτής της τακτικής για συντακτική ανάλυση που βασίζεται σε δύο ενέργειες
 - Διάβασμα ενός token από την είσοδο –**shift**
 - Αναγωγή και σύνδεση ενός ή περισσότερων υποδέντρων σε ένα υπόδεντρο, για μία αντίστοιχη παραγωγή - **reduce**
 - ◆ Τα υπόδεντρα που ενώνονται αποτυπώνουν το δεξί τμήμα (right hand side – RHS) της παραγωγής αυτής
- Ο μηχανισμός αυτός λέγεται **shift-reduce parsing** και μπορεί να υποστηριχθεί με αυτόματα στοίβας
 - *Pushdown automata*



Ανοδική συντακτική ανάλυση (4/30)

Shift-reduce automaton

Αυτόματο στοίβας (push down automaton) ή αλλιώς και shift-reduce automaton:

□ Αποτελείται από:

- Μία στοίβα (μπορεί να περιέχει τερματικά και μη τερματικά σύμβολα)
- Λογική ελέγχου αυτομάτου

□ Μπορεί να κάνει:

○ **Shift**

- Δηλ. *push* το παρόν σύμβολο εισόδου στη στοίβα

○ **Reduce**

- Εάν η ακολουθία συμβόλων β στην κορυφή της στοίβας κάνει match το δεξί τμήμα κάποιας παραγωγής $X \rightarrow \beta$
- Κάνει *pop* τα σύμβολα β από τη στοίβα
- Κάνει *push* το X στη στοίβα

○ **Accept**

Θα κάνουμε αρχικά την παραδοχή
ότι το αυτόματο πάντα κάνει
αναγωγή εάν μπορεί



Ανοδική συντακτική ανάλυση (5/30)

Στοίβα

$expr \rightarrow expr \ op \ expr$
 $expr \rightarrow (expr)$
 $expr \rightarrow -expr$
 $expr \rightarrow num$
 $op \rightarrow + \mid - \mid *$

Όταν κάνουμε match / reduce β σύμβολα στη στοίβα με παραγωγή $X \rightarrow \beta$ και κάνουμε $pop(\beta)$ $push(X)$ θα δημιουργούμε και το υπόδεντρο $X \rightarrow \beta$

num	*	(	num	+	num	)
-----	---	---	-----	---	-----	---

Λέξη εισόδου



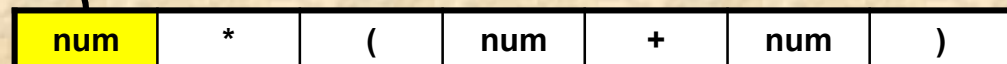
Ανοδική συντακτική ανάλυση (6/30)

Στοίβα



SHIFT

$expr \rightarrow expr \ op \ expr$
 $expr \rightarrow (expr)$
 $expr \rightarrow -expr$
 $expr \rightarrow num$
 $op \rightarrow + \mid - \mid *$





Ανοδική συντακτική ανάλυση (7/30)

Στοίβα

num

SHIFT

$expr \rightarrow expr\ op\ expr$
 $expr \rightarrow (expr)$
 $expr \rightarrow -expr$
 $expr \rightarrow num$
 $op \rightarrow + \mid - \mid *$

*	(	num	+	num	)
---	---	-----	---	-----	---



Ανοδική συντακτική ανάλυση (8/30)

Στοίβα

num

REDUCE

$expr \rightarrow expr\ op\ expr$
 $expr \rightarrow (expr)$
 $expr \rightarrow -expr$
 $expr \rightarrow \mathbf{num}$
 $op \rightarrow + \mid - \mid *$



*	(	num	+	num	)
---	---	-----	---	-----	---



Ανοδική συντακτική ανάλυση (9/30)

Στοίβα



REDUCE

num

$expr \rightarrow expr \ op \ expr$
 $expr \rightarrow (expr)$
 $expr \rightarrow -expr$
 $expr \rightarrow num$
 $op \rightarrow + \mid - \mid *$

*	(	num	+	num	)
---	---	-----	---	-----	---



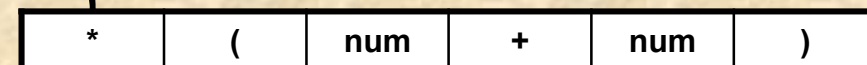
Ανοδική συντακτική ανάλυση (10/30)

Στοίβα



SHIFT

num



$expr \rightarrow expr \ op \ expr$
 $expr \rightarrow (expr)$
 $expr \rightarrow -expr$
 $expr \rightarrow num$
 $op \rightarrow + \mid - \mid *$



Ανοδική συντακτική ανάλυση (11/30)

Στοίβα



SHIFT

num

$expr \rightarrow expr \ op \ expr$
 $expr \rightarrow (expr)$
 $expr \rightarrow -expr$
 $expr \rightarrow num$
 $op \rightarrow + \mid - \mid *$

(	num	+	num	)
---	-----	---	-----	---



Ανοδική συντακτική ανάλυση (12/30)

Στοίβα



REDUCE

num *

$expr \rightarrow expr \ op \ expr$
 $expr \rightarrow (expr)$
 $expr \rightarrow -expr$
 $expr \rightarrow num$
 $op \rightarrow + \mid - \mid *$

(num + num)

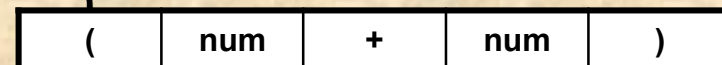
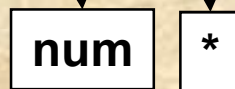


Ανοδική συντακτική ανάλυση (13/30)

Στοίβα



SHIFT

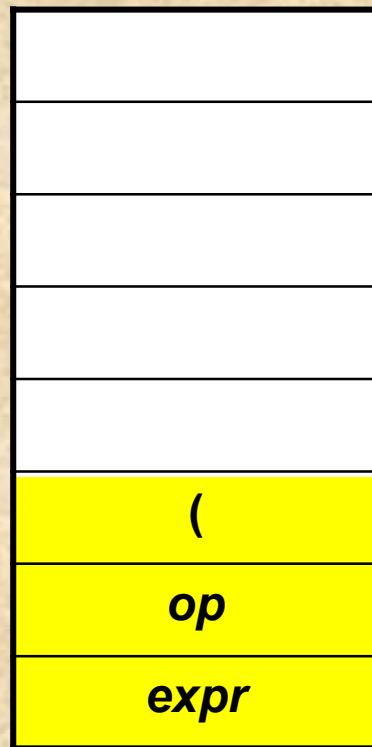


$expr \rightarrow expr \ op \ expr$
 $expr \rightarrow (expr)$
 $expr \rightarrow -expr$
 $expr \rightarrow num$
 $op \rightarrow + \mid - \mid *$

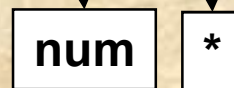


Ανοδική συντακτική ανάλυση (14/30)

Στοίβα



SHIFT



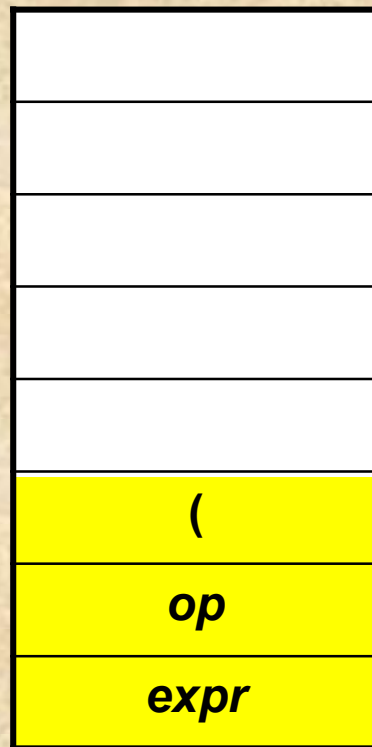
$expr \rightarrow expr\ op\ expr$
 $expr \rightarrow (expr)$
 $expr \rightarrow -expr$
 $expr \rightarrow num$
 $op \rightarrow + \mid - \mid *$





Ανοδική συντακτική ανάλυση (15/30)

Στοίβα



SHIFT

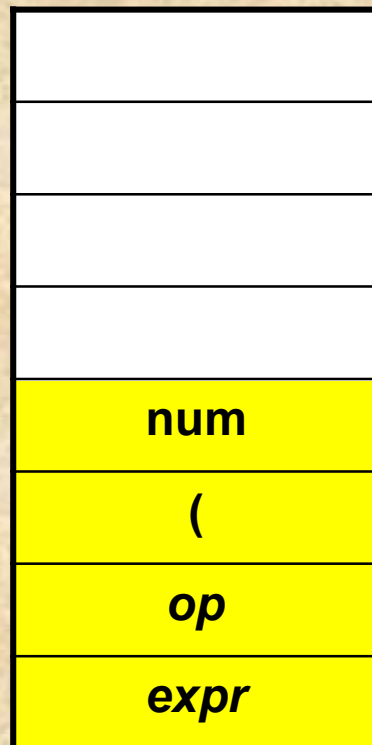


$expr \rightarrow expr\ op\ expr$
 $expr \rightarrow (expr)$
 $expr \rightarrow -expr$
 $expr \rightarrow num$
 $op \rightarrow + \mid - \mid *$

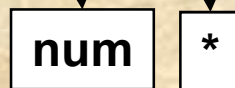


Ανοδική συντακτική ανάλυση (16/30)

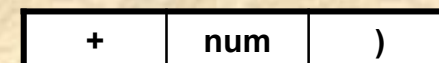
Στοίβα



SHIFT



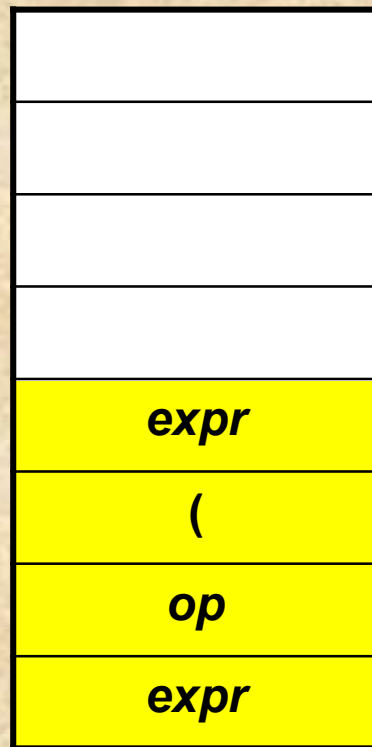
$expr \rightarrow expr \ op \ expr$
 $expr \rightarrow (expr)$
 $expr \rightarrow -expr$
 $expr \rightarrow num$
 $op \rightarrow + \mid - \mid *$



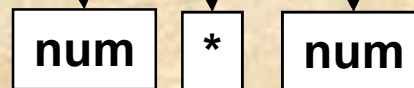


Ανοδική συντακτική ανάλυση (17/30)

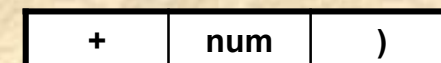
Στοίβα



REDUCE



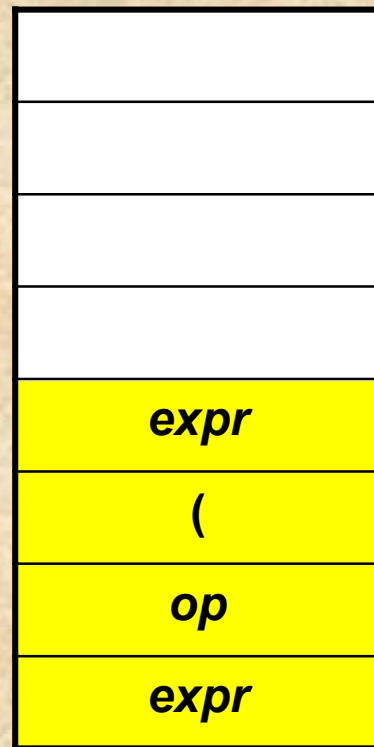
$expr \rightarrow expr \ op \ expr$
 $expr \rightarrow (expr)$
 $expr \rightarrow -expr$
 $expr \rightarrow \mathbf{num}$
 $op \rightarrow + \mid - \mid *$



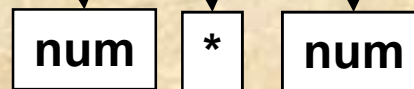


Ανοδική συντακτική ανάλυση (18/30)

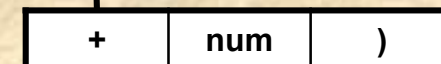
Στοίβα



SHIFT



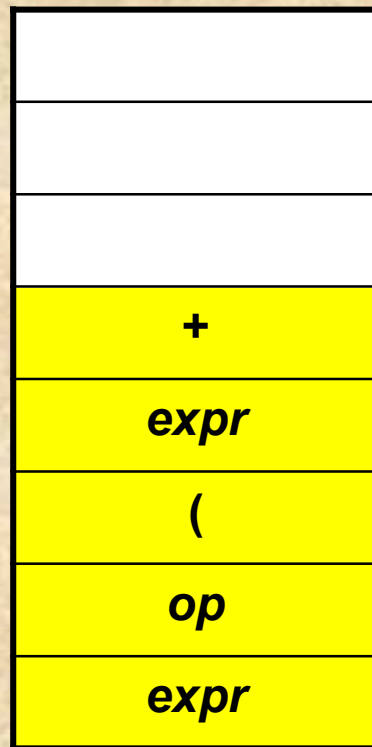
$expr \rightarrow expr \ op \ expr$
 $expr \rightarrow (expr)$
 $expr \rightarrow -expr$
 $expr \rightarrow num$
 $op \rightarrow + \mid - \mid *$





Ανοδική συντακτική ανάλυση (19/30)

Στοίβα



SHIFT

num

*

num

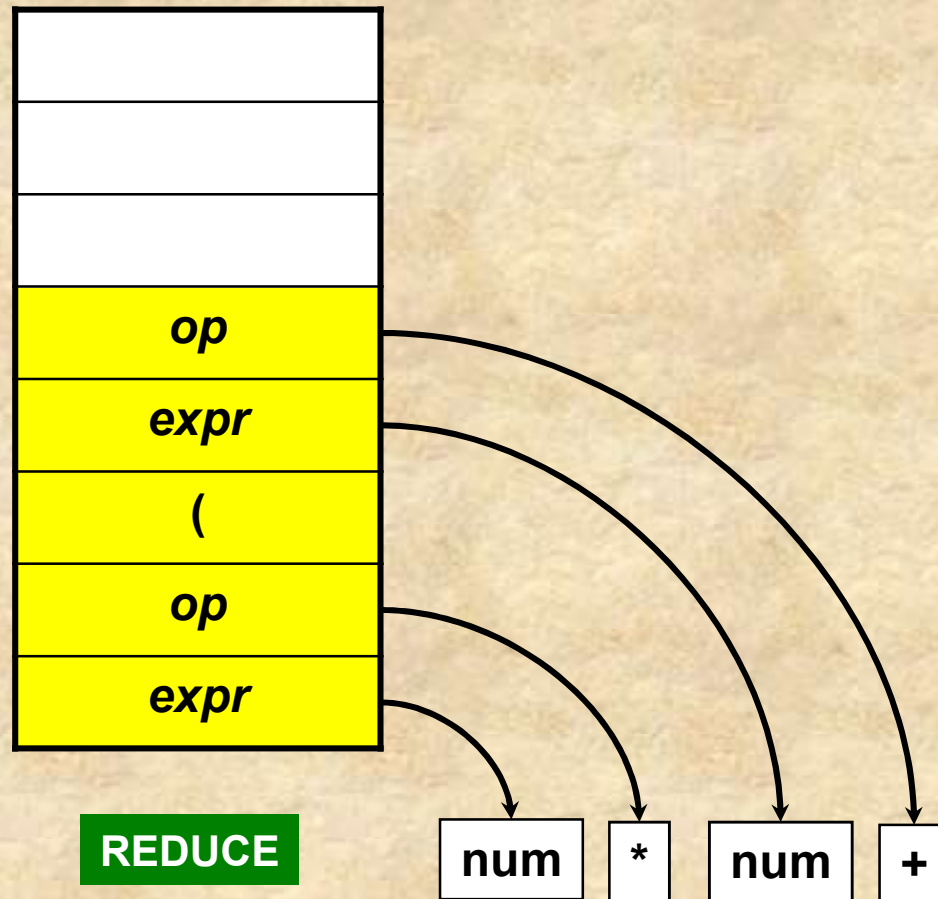
$expr \rightarrow expr \ op \ expr$
 $expr \rightarrow (expr)$
 $expr \rightarrow -expr$
 $expr \rightarrow num$
 $op \rightarrow + \mid - \mid *$

num	)
-----	---



Ανοδική συντακτική ανάλυση (20/30)

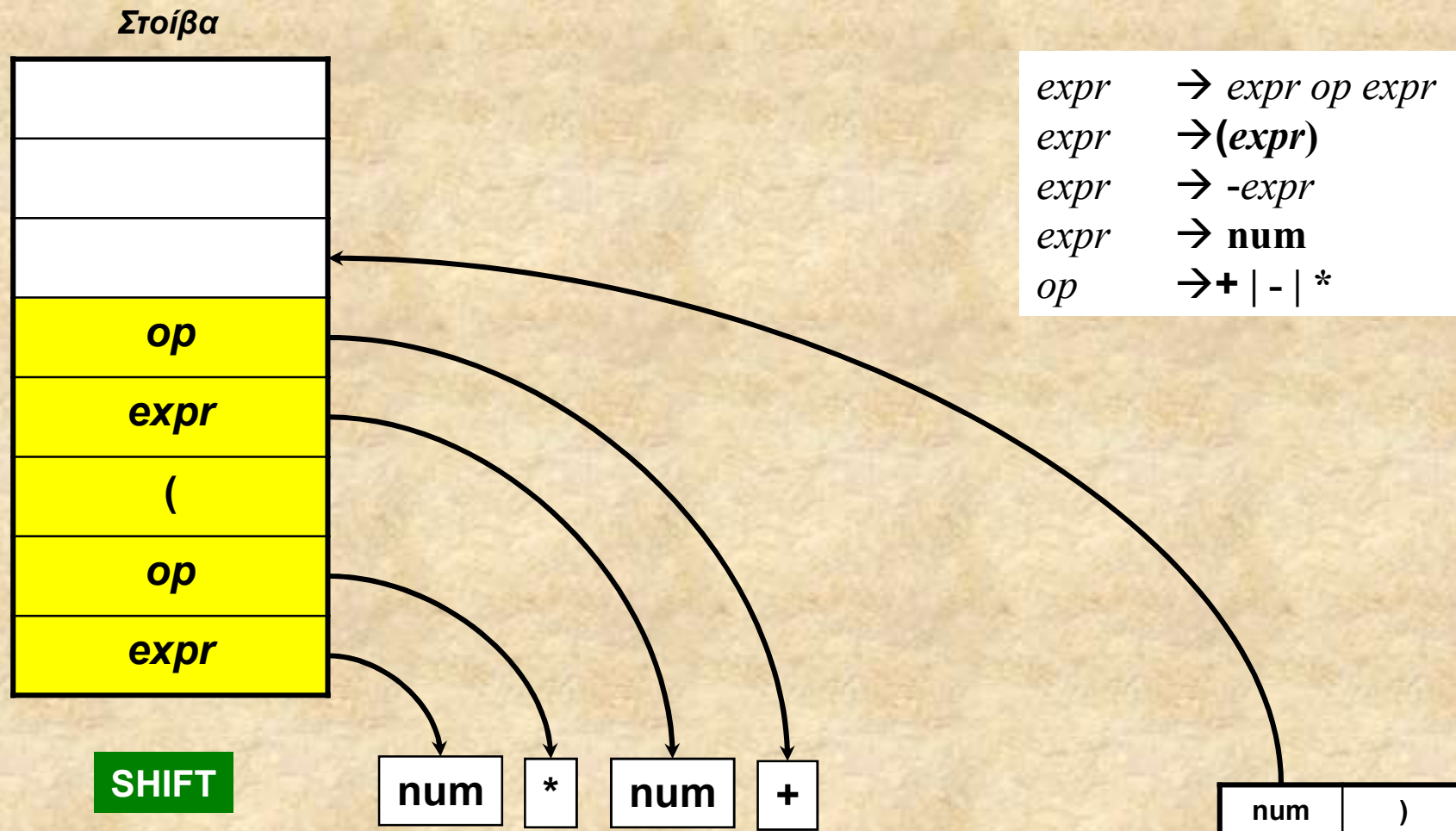
Στοίβα



$expr \rightarrow expr\ op\ expr$
 $expr \rightarrow (expr)$
 $expr \rightarrow -expr$
 $expr \rightarrow num$
 $op \rightarrow + \mid - \mid *$



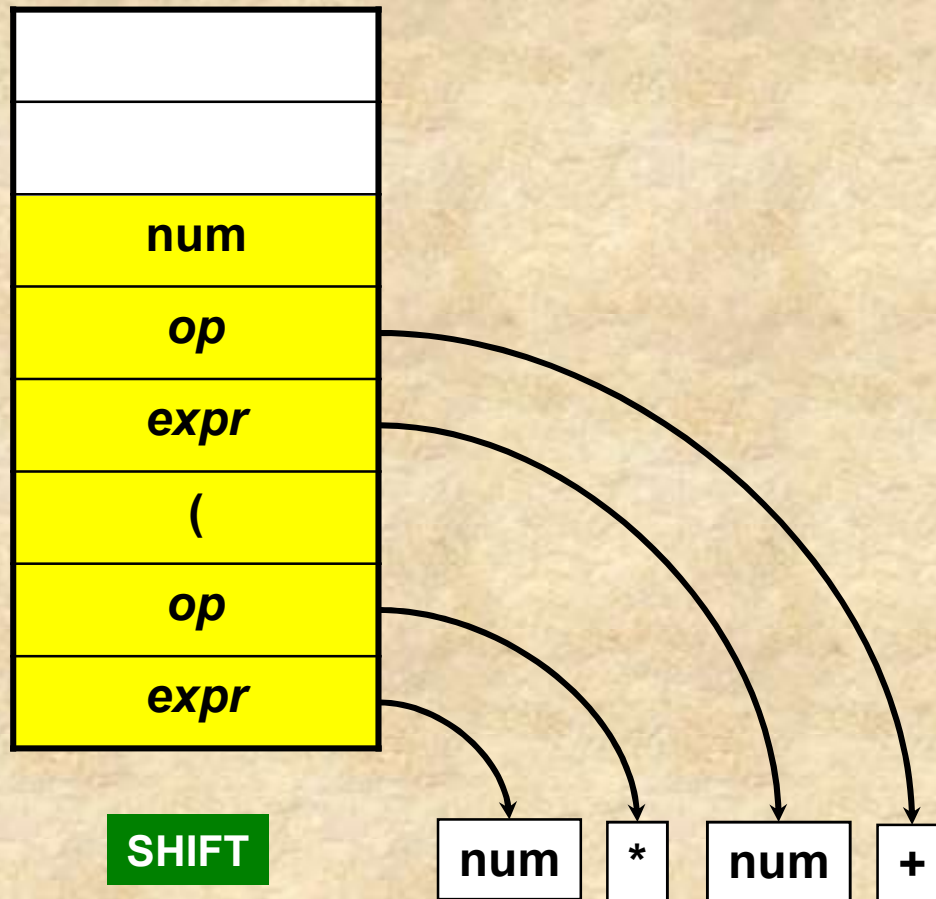
Ανοδική συντακτική ανάλυση (21/30)





Ανοδική συντακτική ανάλυση (22/30)

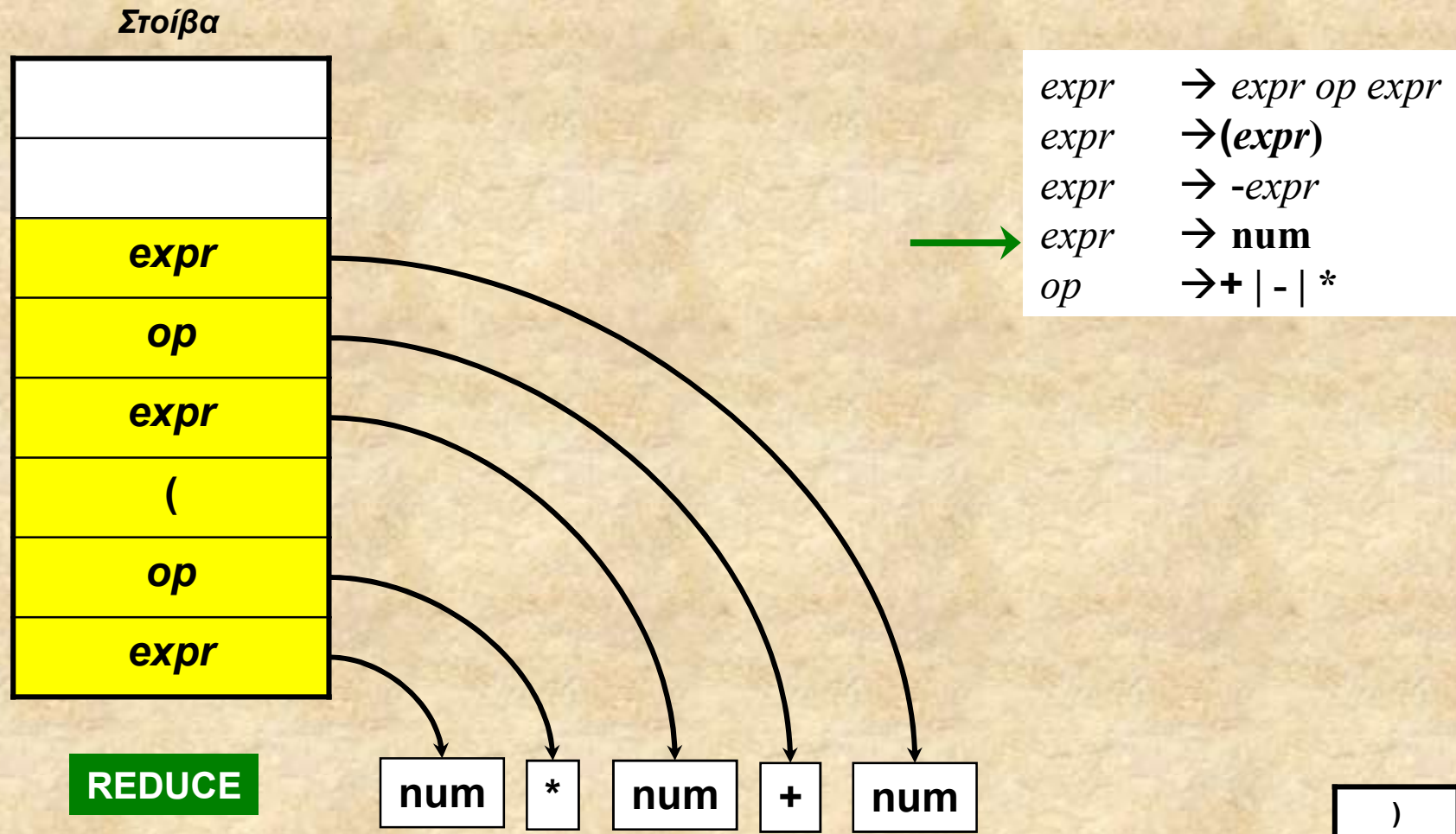
Στοίβα



$expr$	$\rightarrow expr\ op\ expr$
$expr$	$\rightarrow (expr)$
$expr$	$\rightarrow -expr$
$expr$	$\rightarrow num$
op	$\rightarrow + \mid - \mid *$

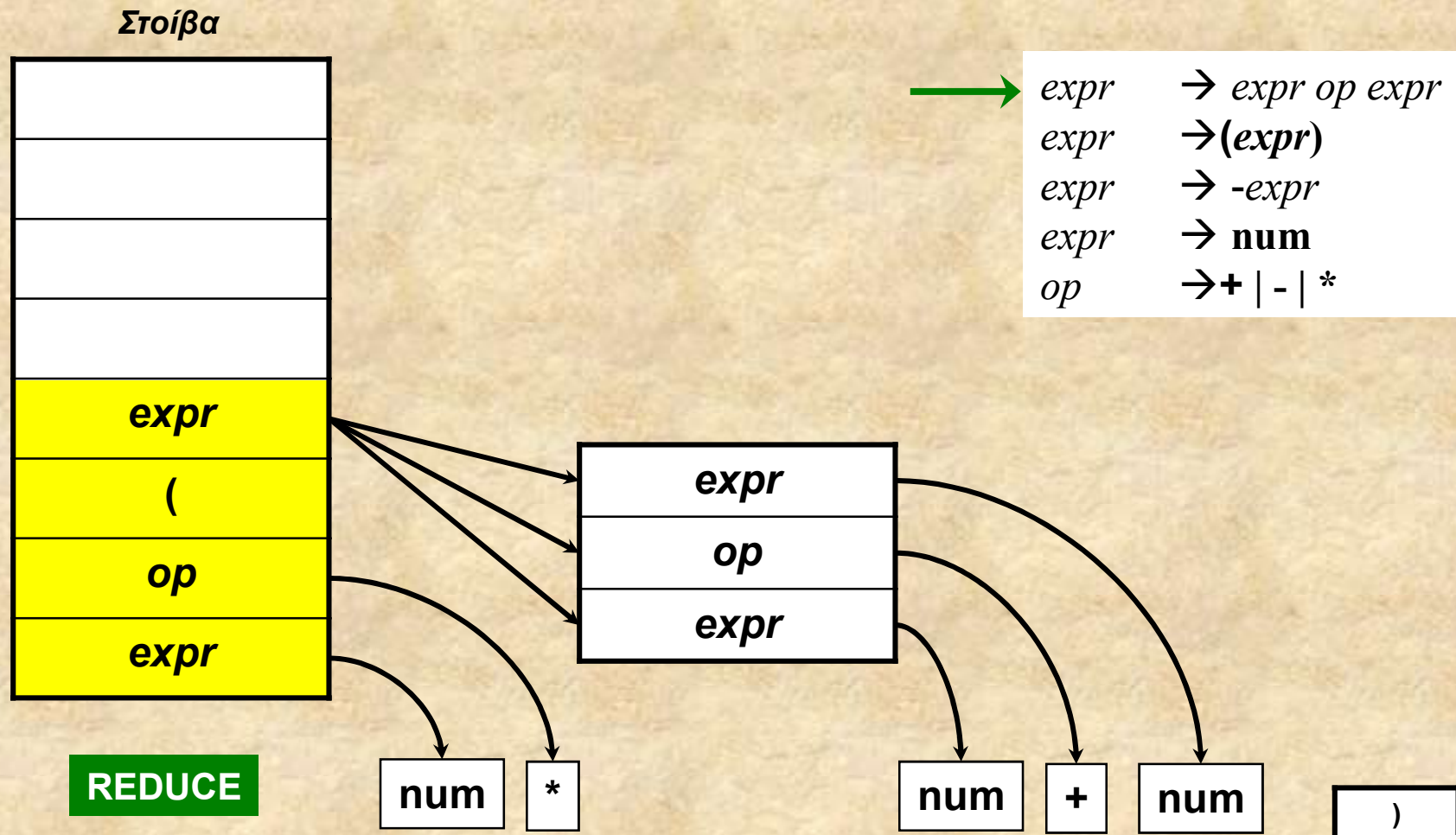


Ανοδική συντακτική ανάλυση (23/30)



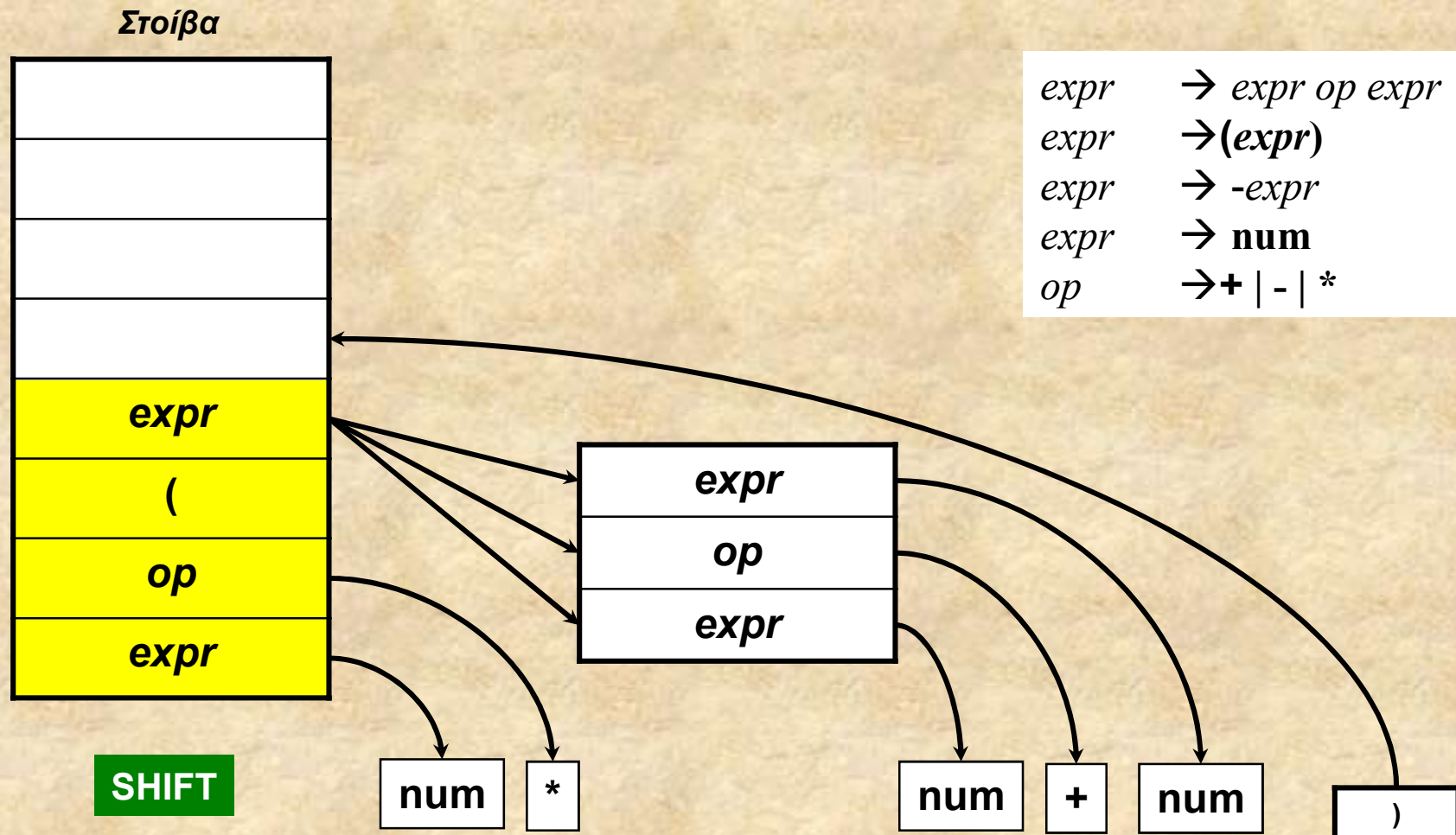


Ανοδική συντακτική ανάλυση (24/30)



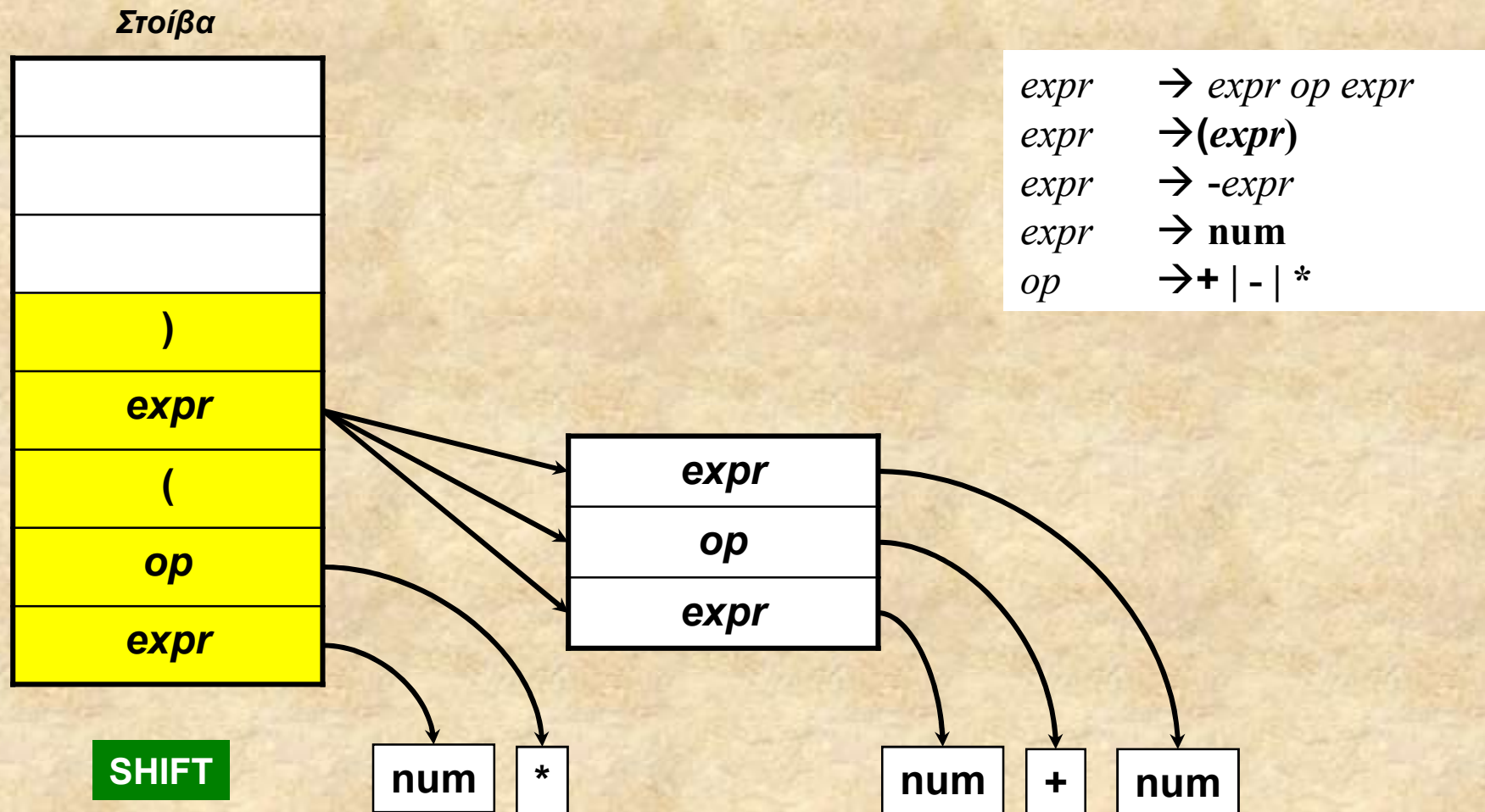


Ανοδική συντακτική ανάλυση (25/30)



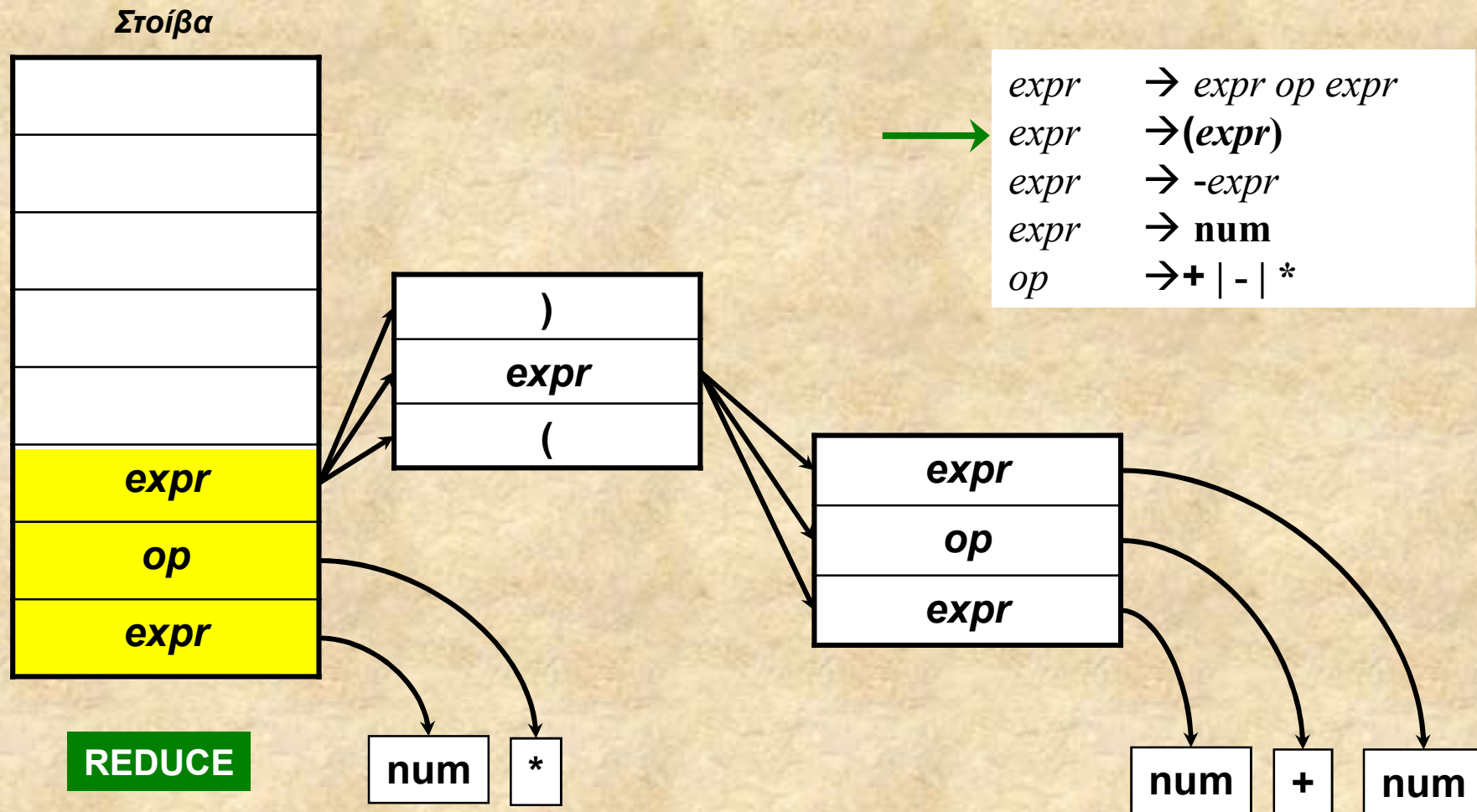


Ανοδική συντακτική ανάλυση (26/30)





Ανοδική συντακτική ανάλυση (27/30)



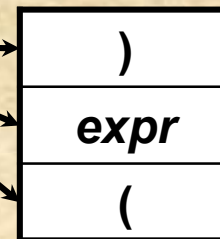
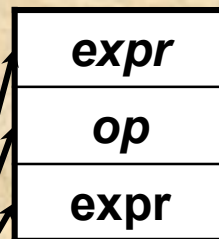


Ανοδική συντακτική ανάλυση (28/30)

Στοίβα

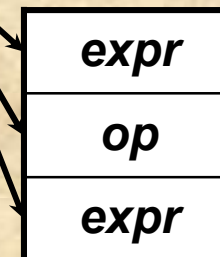


REDUCE



num

*



num

+

num

→ $expr \ op \ expr$
→ $(expr)$
→ $-expr$
→ **num**
→ **+** | **-** | *****



Ανοδική συντακτική ανάλυση (29/30)

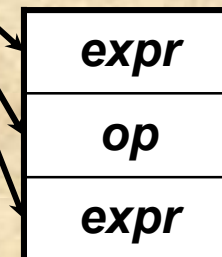
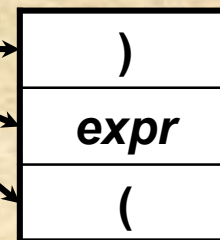
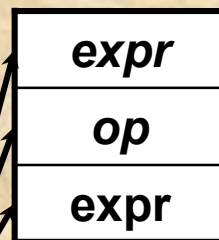
Στοίβα



ACCEPT

Το σύμβολο στη στοίβα είναι το αρχικό σύμβολο της γραμματικής καθώς και η ρίζα του δέντρου συντακτικής ανάλυσης που έχει κατασκευαστεί αυξητικά

$expr \rightarrow expr \ op \ expr$
 $expr \rightarrow (expr)$
 $expr \rightarrow -expr$
 $expr \rightarrow num$
 $op \rightarrow + \mid - \mid *$



num

*

num

+

num



Ανοδική συντακτική ανάλυση (30/30)

- Βασική ιδέα του *shift-reduce parsing*
 - Στόχος είναι η ανακατασκευή του parse tree στηριγμένοι εξολοκλήρου στη λέξη εισόδου
 - Ανάγνωση της εισόδου από *αριστερά* → *δεξιά*
 - Χτίσιμο του δέντρου από κάτω προς τα πάνω αυξητικά
 - Χρήση της στοίβας για την προσωρινή αποθήκευση ακολουθιών τερματικών και μη τερματικών συμβόλων, τα οποία είναι προς αναμονή *γραμματικής αναγωγής - grammar reduction*



Περιεχόμενα

- Αφηρημένα συντακτικά δέντρα
- Ανοδική συντακτική ανάλυση
- *Ασυμφωνίες στην ανοδική ανάλυση*
- Αναλυτές LR



Ασυμφωνίες στην ανοδική ανάλυση (1/28)

Οι πιθανές περιπτώσεις ασυμφωνιών (conflicts) σχετικά με τις δυνατές ενέργειες που μπορούν να εφαρμοστούν σε κάποιο βήμα:

▣ **reduce/reduce conflict**

- Στην κορυφή της στοίβας υπάρχει ακολουθία συμβόλων η οποία κάνει match το δεξί τμήμα περισσότερων της μίας παραγωγής (μη τερματικού συμβόλου)
- Ποια παραγωγή να επιλεγεί για την αναγωγή – *reduction*;

▣ **shift/reduce conflict**

- Στην κορυφή της στοίβας υπάρχει ακολουθία που κάνει match το RHS μίας παραγωγής
- Όμως μπορεί να μην είναι η σωστή επιλογή για αναγωγή
- Αλλά να πρέπει να γίνει *shift* (αντί αναγωγής) και αργότερα να βρεθεί μία διαφορετική αναγωγή



Ασυμφωνίες στην ανοδική ανάλυση (2/28)

Θα δούμε τι περιπτώσεις ασυμφωνιών κατά τη διαδικασία ανοδικής ανάλυσης. Προφανώς οι ασυμφωνίες οφείλονται σε χαρακτηριστικά της γραμματικής. Θα πειράξουμε την αρχική γραμματική ώστε να οδηγήσει σε ασυμφωνίες.

Αρχική γραμματική

$expr \rightarrow expr\ op\ expr$
 $expr \rightarrow (expr)$
 $expr \rightarrow -expr$
 $expr \rightarrow num$
 $op \rightarrow + \mid - \mid *$

Νέα γραμματική

$expr \rightarrow expr\ op\ expr$
 $expr \rightarrow expr - expr$
 $expr \rightarrow (expr)$
 $expr \rightarrow expr -$
 $expr \rightarrow num$
 $op \rightarrow + \mid - \mid *$



Ασυμφωνίες στην ανοδική ανάλυση (3/28)

Στοίβα

<i>expr</i>	$\rightarrow expr\ op\ expr$
<i>expr</i>	$\rightarrow expr - expr$
<i>expr</i>	$\rightarrow (expr)$
<i>expr</i>	$\rightarrow expr-$
<i>expr</i>	$\rightarrow num$
<i>op</i>	$\rightarrow + \mid - \mid *$

num	-	num
-----	---	-----

Λέξη εισόδου



Ασυμφωνίες στην ανοδική ανάλυση (4/28)

Στοίβα



SHIFT

$expr$	$\rightarrow expr\ op\ expr$
$expr$	$\rightarrow expr - expr$
$expr$	$\rightarrow (expr)$
$expr$	$\rightarrow expr -$
$expr$	$\rightarrow num$
op	$\rightarrow + \mid - \mid *$





Ασυμφωνίες στην ανοδική ανάλυση (5/28)

Στοίβα

num

SHIFT

<i>expr</i>	$\rightarrow expr\ op\ expr$
<i>expr</i>	$\rightarrow expr - expr$
<i>expr</i>	$\rightarrow (expr)$
<i>expr</i>	$\rightarrow expr -$
<i>expr</i>	$\rightarrow num$
<i>op</i>	$\rightarrow + \mid - \mid *$

-	num
---	-----



Ασυμφωνίες στην ανοδική ανάλυση (6/28)

Στοίβα



REDUCE

num

$expr \rightarrow expr \ op \ expr$
 $expr \rightarrow expr - expr$
 $expr \rightarrow (expr)$
 $expr \rightarrow expr -$
 $expr \rightarrow num$
 $op \rightarrow + \mid - \mid *$

-	num
---	-----



Ασυμφωνίες στην ανοδική ανάλυση (7/28)

Στοίβα



SHIFT

num

<i>expr</i>	$\rightarrow expr\ op\ expr$
<i>expr</i>	$\rightarrow expr - expr$
<i>expr</i>	$\rightarrow (expr)$
<i>expr</i>	$\rightarrow expr -$
<i>expr</i>	$\rightarrow num$
<i>op</i>	$\rightarrow + \mid - \mid *$





Ασυμφωνίες στην ανοδική ανάλυση (8/28)

Στοίβα



Τρεις δυνατές επιλογές όπως φαίνεται παρακάτω. Ας δούμε τι συμβαίνει εάν επιλέξουμε την πρώτη αναγωγή (1)

1. reduce
2. reduce
3. shift

<i>expr</i>	$\rightarrow expr\ op\ expr$
<i>expr</i>	$\rightarrow expr - expr$
<i>expr</i>	$\rightarrow (expr)$
<i>expr</i>	$\rightarrow expr -$
<i>expr</i>	$\rightarrow num$
<i>op</i>	$\rightarrow + \mid - \mid *$

num

num



Ασυμφωνίες στην ανοδική ανάλυση (9/28)

Στοίβα



REDUCE



num

$expr \rightarrow expr \ op \ expr$
 $expr \rightarrow expr - expr$
 $expr \rightarrow (expr)$
 $expr \rightarrow expr -$
 $expr \rightarrow num$
 $op \rightarrow + \mid - \mid *$



Ασυμφωνίες στην ανοδική ανάλυση (10/28)

Στοίβα



SHIFT



num

<i>expr</i>	$\rightarrow expr\ op\ expr$
<i>expr</i>	$\rightarrow expr - expr$
<i>expr</i>	$\rightarrow (expr)$
<i>expr</i>	$\rightarrow expr -$
<i>expr</i>	$\rightarrow num$
<i>op</i>	$\rightarrow + \mid - \mid *$

num



Ασυμφωνίες στην ανοδική ανάλυση (11/28)

Στοίβα



SHIFT

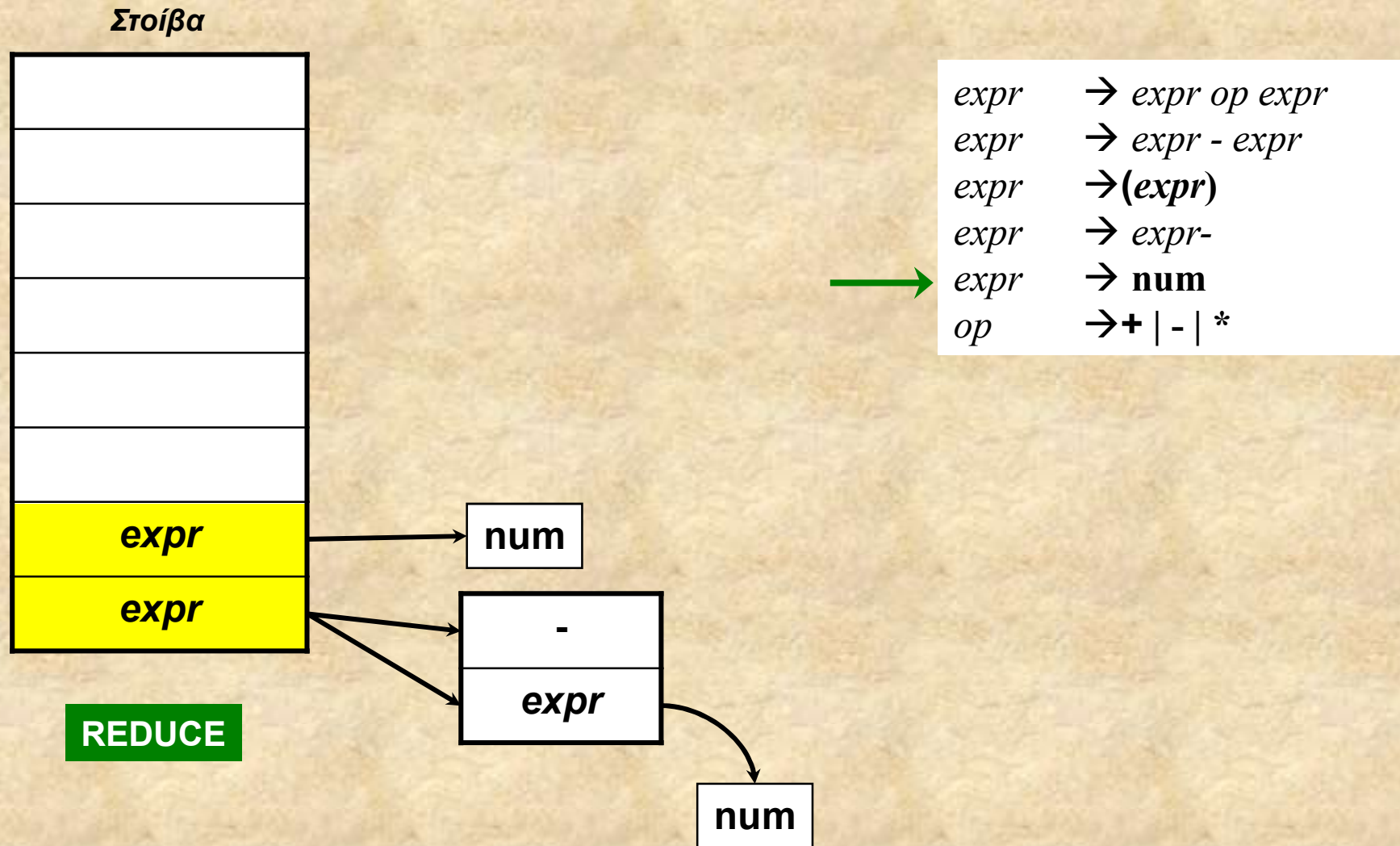


num

<i>expr</i>	$\rightarrow expr\ op\ expr$
<i>expr</i>	$\rightarrow expr - expr$
<i>expr</i>	$\rightarrow (expr)$
<i>expr</i>	$\rightarrow expr -$
<i>expr</i>	$\rightarrow num$
<i>op</i>	$\rightarrow + \mid - \mid *$



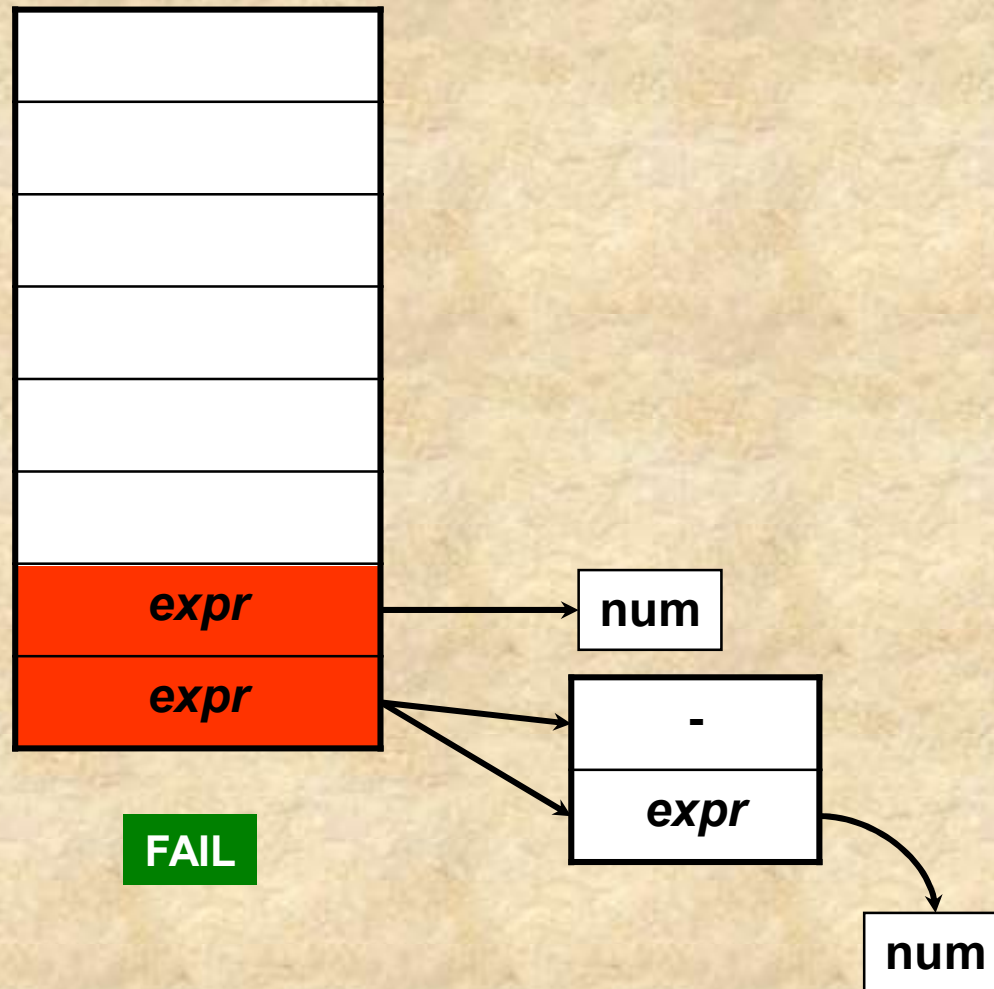
Ασυμφωνίες στην ανοδική ανάλυση (12/28)





Ασυμφωνίες στην ανοδική ανάλυση (13/28)

Στοίβα



$expr \rightarrow expr \text{ op } expr$
 $expr \rightarrow expr - expr$
 $expr \rightarrow (expr)$
 $expr \rightarrow expr -$
 $expr \rightarrow num$
 $op \rightarrow + \mid - \mid *$

Παρατήρηση: σημειώνουμε ότι η επιλογή του κανόνα $expr \rightarrow expr -$ δεν οδήγησε σε «βιώσιμο» δέντρο συντακτικής ανάλυσης.

Αυτό σημαίνει ότι δεν θα κατηγορήσουμε αργότερα αυτόν τον γραμματικό κανόνα ως γεννήτορα διαφορούμενης ανάλυσης για τη συγκεκριμένη λέξη εισόδου, αφού κάτι τέτοιο θα απαιτούσε να οδηγηθούμε σε ένα εφικτό συντακτικό δέντρο, κάτι που όμως δε συνέβηκε.



Ασυμφωνίες στην ανοδική ανάλυση (14/28)

Στοίβα



Ας δοκιμάσουμε με τη δεύτερη αναγωγή (2)

1. reduce
2. reduce
3. shift

<i>expr</i>	$\rightarrow expr\ op\ expr$
<i>expr</i>	$\rightarrow expr - expr$
<i>expr</i>	$\rightarrow (expr)$
<i>expr</i>	$\rightarrow expr -$
<i>expr</i>	$\rightarrow num$
<i>op</i>	$\rightarrow + \mid - \mid *$

num

num



Ασυμφωνίες στην ανοδική ανάλυση (15/28)

Στοίβα



REDUCE

num

-

num

$expr \rightarrow expr\ op\ expr$
 $expr \rightarrow expr - expr$
 $expr \rightarrow (expr)$
 $expr \rightarrow expr -$
 $expr \rightarrow num$
 $op \rightarrow + \mid - \mid *$



Ασυμφωνίες στην ανοδική ανάλυση (16/28)

Στοίβα



SHIFT

num

-

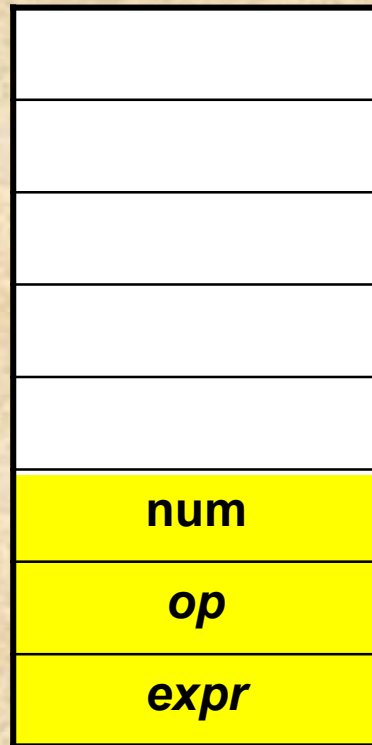
num

<i>expr</i>	$\rightarrow expr\ op\ expr$
<i>expr</i>	$\rightarrow expr - expr$
<i>expr</i>	$\rightarrow (expr)$
<i>expr</i>	$\rightarrow expr -$
<i>expr</i>	$\rightarrow num$
<i>op</i>	$\rightarrow + \mid - \mid *$



Ασυμφωνίες στην ανοδική ανάλυση (17/28)

Στοίβα



SHIFT

num

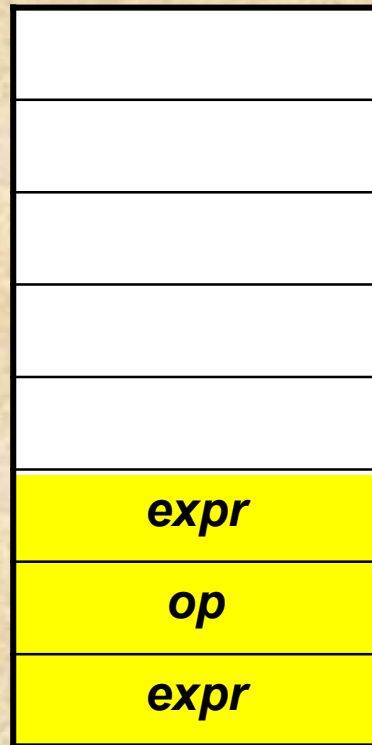
-

<i>expr</i>	$\rightarrow expr\ op\ expr$
<i>expr</i>	$\rightarrow expr - expr$
<i>expr</i>	$\rightarrow (expr)$
<i>expr</i>	$\rightarrow expr -$
<i>expr</i>	$\rightarrow num$
<i>op</i>	$\rightarrow + \mid - \mid *$



Ασυμφωνίες στην ανοδική ανάλυση (18/28)

Στοίβα



REDUCE

num

-

num

$expr \rightarrow expr\ op\ expr$
 $expr \rightarrow expr - expr$
 $expr \rightarrow (expr)$
 $expr \rightarrow expr -$
 $expr \rightarrow num$
 $op \rightarrow + \mid - \mid *$

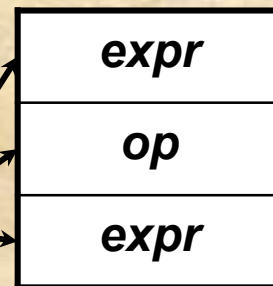


Ασυμφωνίες στην ανοδική ανάλυση (19/28)

Στοίβα



REDUCE



num

-

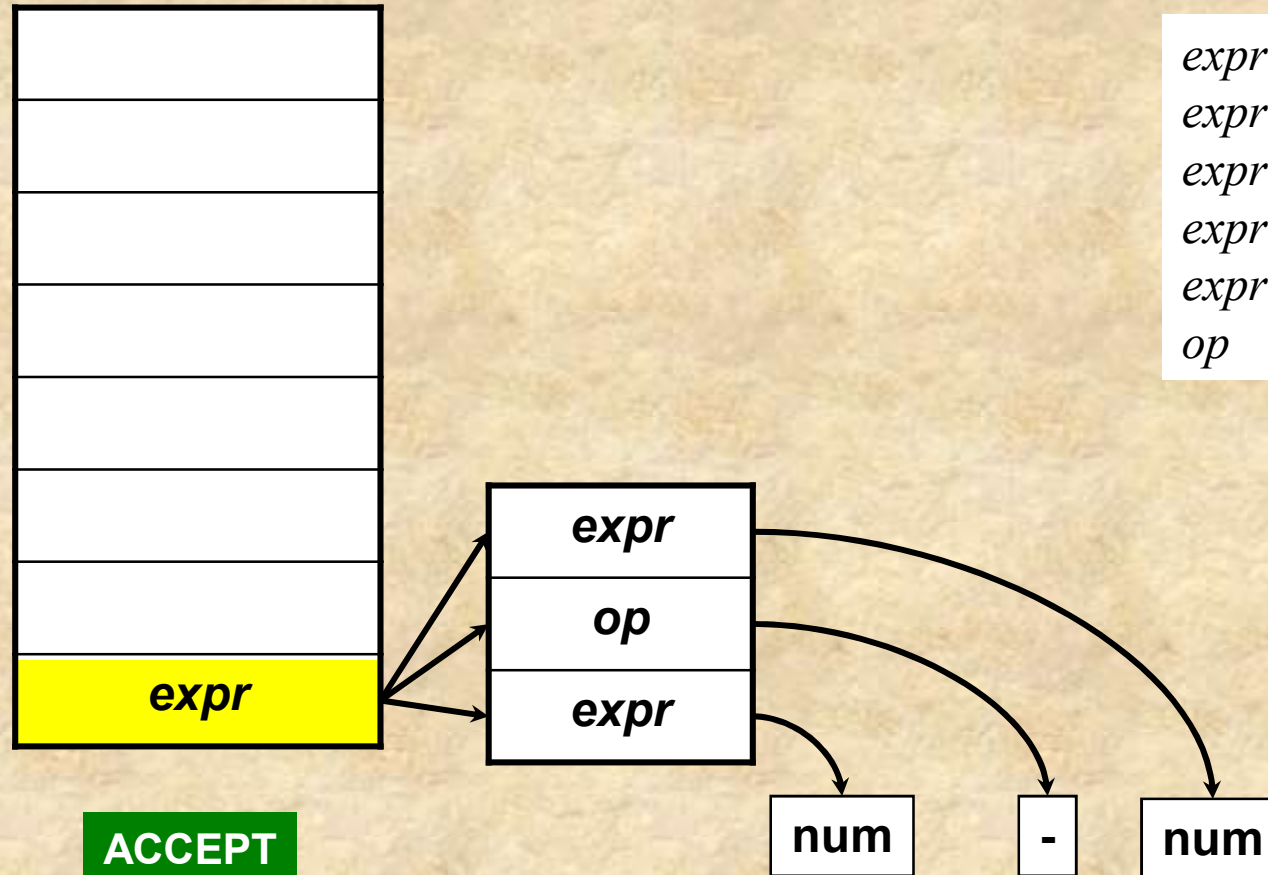
num

→ $expr \ op \ expr$
→ $expr - expr$
→ $(expr)$
→ $expr -$
→ **num**
→ **+** | **-** | *****



Ασυμφωνίες στην ανοδική ανάλυση (20/28)

Στοίβα



<i>expr</i>	$\rightarrow expr\ op\ expr$
<i>expr</i>	$\rightarrow expr - expr$
<i>expr</i>	$\rightarrow (expr)$
<i>expr</i>	$\rightarrow expr -$
<i>expr</i>	$\rightarrow num$
<i>op</i>	$\rightarrow + \mid - \mid *$



Ασυμφωνίες στην ανοδική ανάλυση (21/28)

Στοίβα



Ας δοκιμάσουμε με να κάνουμε
shift (3)

1. reduce
2. reduce
3. shift

<i>expr</i>	$\rightarrow expr\ op\ expr$
<i>expr</i>	$\rightarrow expr - expr$
<i>expr</i>	$\rightarrow (expr)$
<i>expr</i>	$\rightarrow expr -$
<i>expr</i>	$\rightarrow num$
<i>op</i>	$\rightarrow + \mid - \mid *$

num

num



Ασυμφωνίες στην ανοδική ανάλυση (22/28)

Στοίβα



SHIFT

num

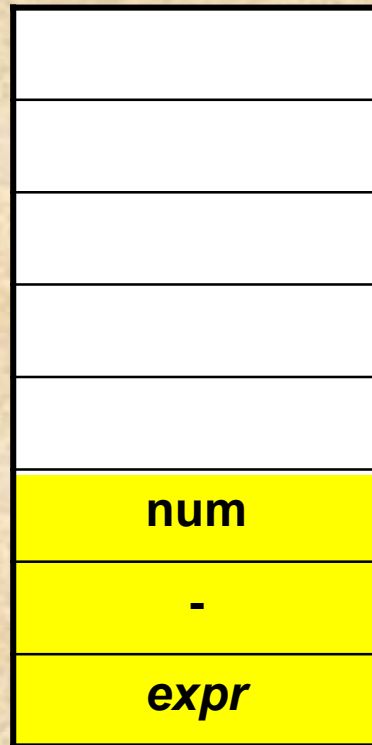
<i>expr</i>	$\rightarrow expr\ op\ expr$
<i>expr</i>	$\rightarrow expr - expr$
<i>expr</i>	$\rightarrow (expr)$
<i>expr</i>	$\rightarrow expr -$
<i>expr</i>	$\rightarrow num$
<i>op</i>	$\rightarrow + \mid - \mid *$

num



Ασυμφωνίες στην ανοδική ανάλυση (23/28)

Στοίβα



SHIFT

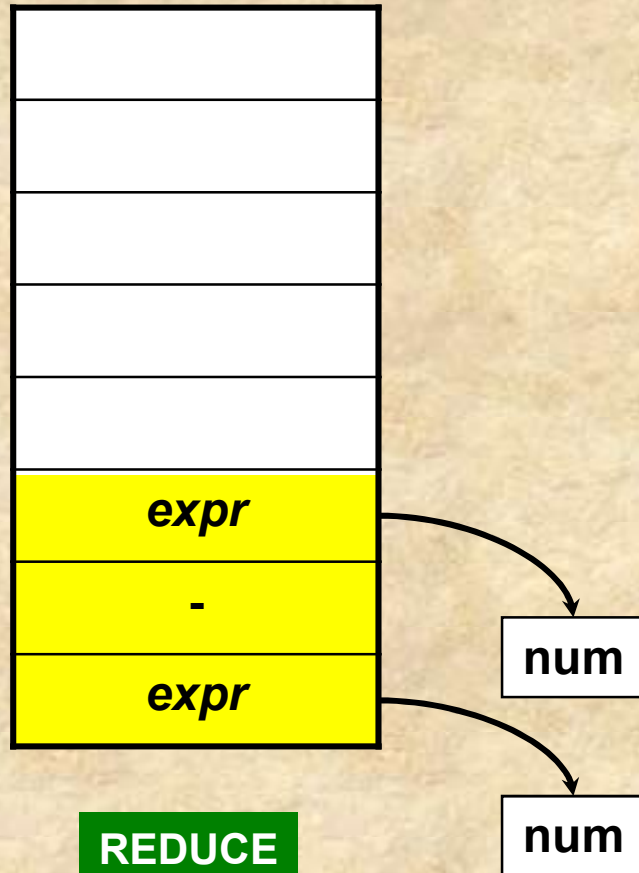
num

<i>expr</i>	$\rightarrow expr\ op\ expr$
<i>expr</i>	$\rightarrow expr - expr$
<i>expr</i>	$\rightarrow (expr)$
<i>expr</i>	$\rightarrow expr -$
<i>expr</i>	$\rightarrow num$
<i>op</i>	$\rightarrow + \mid - \mid *$



Ασυμφωνίες στην ανοδική ανάλυση (24/28)

Στοίβα



$expr \rightarrow expr \ op \ expr$
 $expr \rightarrow expr - expr$
 $expr \rightarrow (expr)$
 $expr \rightarrow expr -$
 $expr \rightarrow num$
 $op \rightarrow + \mid - \mid *$

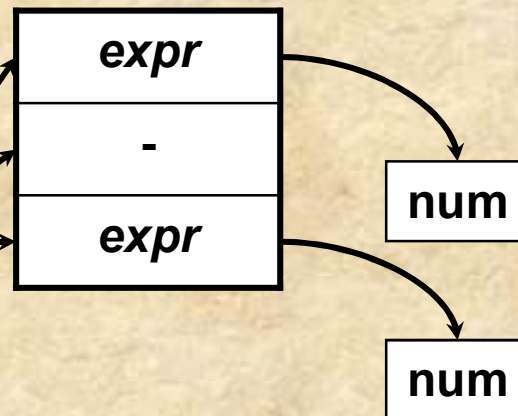


Ασυμφωνίες στην ανοδική ανάλυση (25/28)

Στοίβα



REDUCE

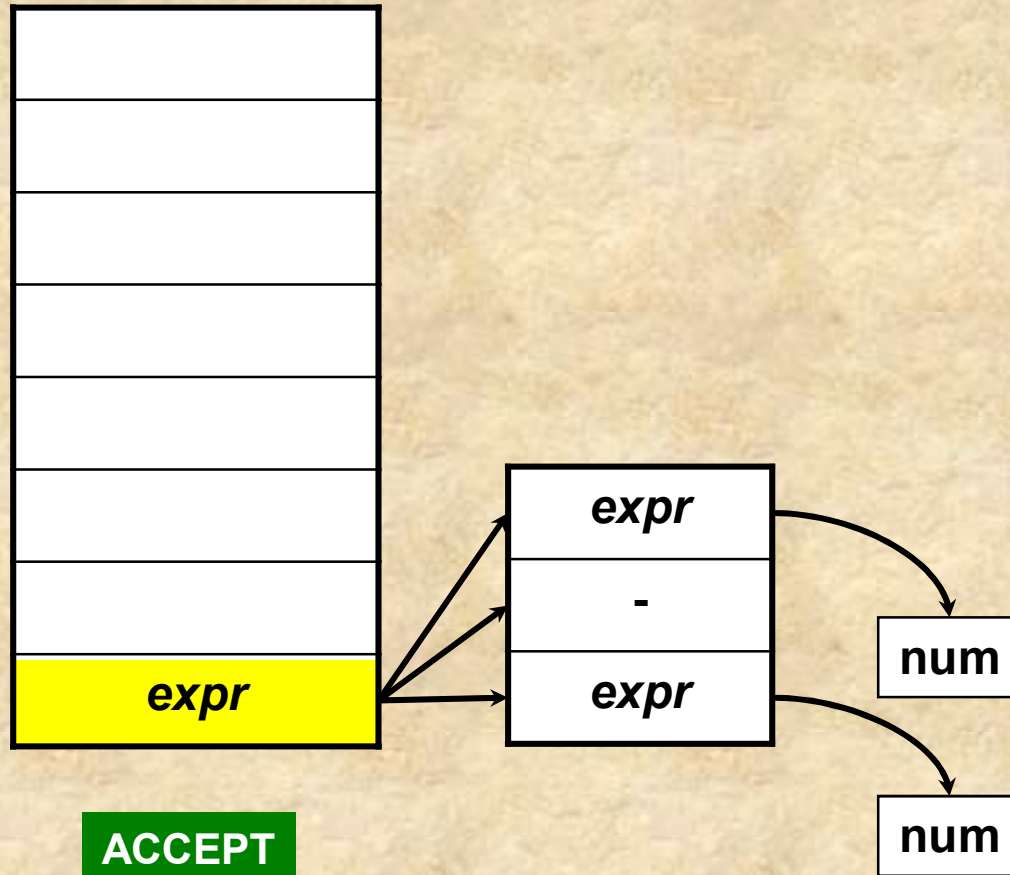


$expr \rightarrow expr \ op \ expr$
 $expr \rightarrow expr - expr$
 $expr \rightarrow (expr)$
 $expr \rightarrow expr -$
 $expr \rightarrow num$
 $op \rightarrow + \mid - \mid *$



Ασυμφωνίες στην ανοδική ανάλυση (26/28)

Στοίβα

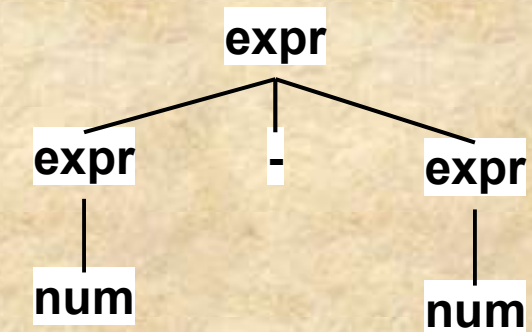
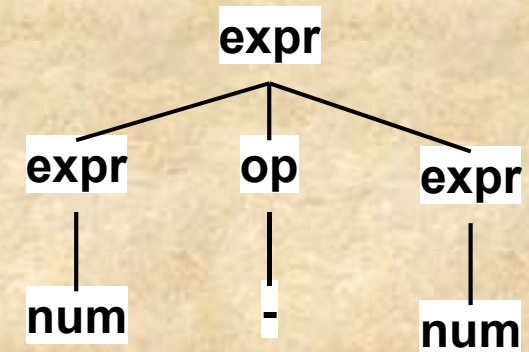


<i>expr</i>	$\rightarrow expr\ op\ expr$
<i>expr</i>	$\rightarrow expr - expr$
<i>expr</i>	$\rightarrow (expr)$
<i>expr</i>	$\rightarrow expr-$
<i>expr</i>	$\rightarrow num$
<i>op</i>	$\rightarrow + \mid - \mid *$



Ασυμφωνίες στην ανοδική ανάλυση – Ένθετο 1

- Το γεγονός ότι για την ίδια λέξη εισόδου καταλήξαμε σε δύο διαφορετικά δέντρα συντακτικής ανάλυσης (όπως αυτά αποτυπώνονται στις «συνδέσεις» των αναγωγών στη στοίβα), αποδεικνύει ότι η γραμματική είναι διφορούμενη
- Για το φαινόμενο αυτό πρέπει να κατηγορήσουμε κάποιον από τους νεοεισαχθέντες γραμματικούς κανόνες
- Επίσης όπως βλέπουμε *το πρόβλημα της διφορούμενης ανάλυσης εμφανίζεται στον αναλυτή ως ένα shift / reduce conflict*
- Το reduce/reduce conflict στην προκειμένη περίπτωση δεν δημιουργεί τέτοιο πρόβλημα, απλώς *ενδέχεται, με το ένα reduce να μην εντοπίσουμε το εφικτό συντακτικό δέντρο*, δηλ. χρειαζόμαστε κάποιο κριτήριο επιλογής ώστε να οδηγούμαστε στο «καλό» reduce





Ασυμφωνίες στην ανοδική ανάλυση (27/28)

Το **shift/reduce** conflict για τον κανόνα $op \rightarrow -$ οφείλεται σε διαφορετική ερμηνεία στην οποία οδηγεί η γραμματική (ambiguity). Ειδικότερα, ο αναλυτής θα μπορούσε να μην κάνει τώρα *reduce*, αλλά να κάνει *shift* και αργότερα *reduce* με τον κανόνα $expr \rightarrow expr - expr$.

Αυτό όμως οδηγεί πάντα σε δύο διαφορετικά υπόδεντρα για κάθε εμπλοκή του δυαδικού $-$, καθώς στο ένα έχουμε μόνο την παραγωγή $expr \rightarrow expr - expr$ ενώ στο άλλο το $expr \rightarrow expr op expr$ και $op \rightarrow -$. Η εξάλειψη της διαφορετικής ανάλυσης γίνεται με αφαίρεση του $expr \rightarrow expr - expr$.



Ασυμφωνίες στην ανοδική ανάλυση (28/28)

Εκτός ύλης

Η εξάλειψη του **reduce/reduce** conflict για τον κανόνα $expr \rightarrow expr-$ και $op \rightarrow -$ γίνεται με τη χρήση ενός look-ahead για τα σύμβολα εισόδου ως εξής:

- Εάν το look-ahead μετά το $-$ προμηνύει $expr$, ο αναλυτής επιλέγει *reduce* με $op \rightarrow -$ γιατί προβλέπει σωστά ότι θα γίνει αργότερα *reduction* μέσω του κανόνα $expr \rightarrow expr\ op\ expr$.
- Ειδιάλλως κάνει απευθείας *reduce* με τον κανόνα $expr \rightarrow expr-$.

Έτσι, εάν έχουμε τη λέξη **num-+num**, ο αναλυτής έχοντας διαβάσει ήδη το **num-** και με look-ahead το $+$ επιλέγει σωστά να κάνει *reduce* $expr \rightarrow expr-$ και να θεωρήσει το **num** σαν $expr$ (αντιπροσωπεύει postfix χρήση του μοναδιαίου $-$).

Αντιθέτως, για τη λέξη εισόδου **num-(num)**, έχοντας ως look-ahead σύμβολο το $($ γνωρίζει ότι αυτό προμηνύει $expr$, άρα κάνει *reduce* $op \rightarrow -$ ώστε να έχει στη στοίβα $expr\ op$.



Ασυμφωνίες στην ανοδική ανάλυση – Ένθετο 2

Εκτός ύλης

Μπορεί να τυποποιηθεί αλγοριθμικά (μηχανιστικά) η προηγούμενη τακτική επίλυσης της ασυμφωνίας (conflict resolution), ώστε να ενσωματώνεται στον αλγόριθμο του s/r αυτομάτου; Ακολουθεί ένας τέτοιος αλγόριθμος σε αντιπαράθεση με το συγκεκριμένο παράδειγμα.

IF

After the reduction by $A \rightarrow \gamma$, e.g., $op \rightarrow -$, the top of the stack matches partially the RHS βA of a grammar rule $B \rightarrow \beta A \delta$, e.g. $expr\ op$ of $expr \rightarrow expr\ op\ expr$

THEN

IF

The look ahead token belongs to the FIRST set of the grammar symbol δ , e.g. $expr$, just following the matched RHS

THEN

Reduce by $A \rightarrow \gamma$, e.g. $op \rightarrow -$

ELSE

Reduce by the other rule, e.g. $expr \rightarrow expr-$



Περιεχόμενα

- Αφηρημένα συντακτικά δέντρα
- Ανοδική συντακτική ανάλυση
- Ασυμφωνίες στην ανοδική ανάλυση
- *Αναλυτές LR*



Αναλυτές LR (1/14)

- Ένας αναλυτής $LR(k)$ είναι ένας ανοδικός αναλυτής με τα εξής χαρακτηριστικά
 - L , για left→right ανάγνωση των συμβόλων εισόδου
 - R , για την ανάδρομη κατασκευή (από το τέλος προς την αρχή) ενός rightmost derivation
 - k , το πλήθος των απαιτούμενων look-ahead συμβόλων για την ανάλυση – εάν δεν αναφέρεται το k τότε θεωρείται ότι είναι 1



Αναλυτές LR (2/14)

- Η τεχνική LR parsing είναι πολύ σημαντική για τους παρακάτω λόγους:
 - Μπορούν να κατασκευαστούν για **σχεδόν όλες** τις προγραμματιστικές δομές που αντιστοιχούν σε context-free grammars
 - Η LR μέθοδος είναι η πιο γενική shift-reduce και χωρίς backtracking μέθοδος συντακτικής ανάλυσης, ενώ υλοποιείται το ίδιο αποτελεσματικά όσο άλλες αντίστοιχες μέθοδοι
 - Η οικογένεια των γραμματικών που γίνονται parsed μέσω LR μεθόδων είναι γνήσιο υπερσύνολο της οικογένειας LL (των γραμματικών που αναγνωρίζονται από predictive top-down parses)
 - Ένας LR parser μπορεί να εντοπίσει ένα συντακτικό λάθος το συντομότερο δυνατό με left→right ανάγνωση εισόδου



Αναλυτές LR (3/14)

- Το κύριο μειονέκτημα των LR parsers είναι πως η κατασκευή τους από την αρχή για μία φυσιολογική γραμματική γλώσσας-προγραμματισμού είναι πολύ δύσκολη
- Απαιτείται ειδικό εργαλείο ανάπτυξης, μία γεννήτρια LR αναλυτών – LR parser generator, π.χ. ο YACC
- Υπάρχουν τρεις τρόποι κατασκευής του parsing table ενός LR parser
 - Simple LR (SLR), η απλούστερη αλλά λιγότερο ισχυρή (*αυτή θα μελετήσουμε μόνο*)
 - Canonical LR, η ισχυρότερη και κατασκευαστικά πολυπλοκότερη
 - Look-ahead LR (LALR), ενδιάμεση σε ισχύ και κόστος ανάπτυξης
- Θα δούμε τη βασική δομή και τον τρόπο (αλγόριθμο) λειτουργίας ενός LR parser



Αναλυτές LR (4/14)

Handle(γ), για έναν δεξιό προτασιακό τύπο γ (δηλ. ακολουθία συμβόλων που παράγεται από το αρχικό σύμβολο μόνο με δεξιές παραγωγές).

- Είναι μία παραγωγή της μορφής $A \rightarrow \beta$ καθώς και μία συγκεκριμένη θέση στο γ στην οποία βρίσκεται το β ,
- όπου εάν αντικαταστήσουμε το β με το A (δηλ. εφαρμόσουμε την παραγωγή ανάποδα – αναγωγή ή reduction), λαμβάνουμε την ακριβώς προηγούμενη δεξιά παραγωγή του γ .

handle(γ) = παραγωγή και θέση αυτής ώστε να πάμε στον προηγούμενο του γ προτασιακό τύπο

Παρατήρηση: οι δεξιές παραγωγές παράγουν την λέξη τερματικών συμβόλων από τα δεξιά προς τα αριστερά!

Εκτός ύλης



Αναλυτές LR (5/14)

Προσδιορισμός δεξιάς παραγωγής από λέξη εισόδου. Έστω ότι έχουμε τη λέξη εισόδου w . Το ζητούμενο είναι να βρεθεί η ακολουθία δεξιών παραγωγών της παρακάτω μορφής που οδηγεί στο w :

$$S = \gamma_0 \Rightarrow \text{rm } \gamma_1 \Rightarrow \text{rm } \gamma_2 \dots \Rightarrow \text{rm } \gamma_{n-1} \Rightarrow \text{rm } \gamma_n = w$$

Αυτό γίνεται ως εξής: έστω β_n handle του γ_n με αντίστοιχη παραγωγή $A_n \rightarrow \beta_n$. Αντικαθιστούμε το β_n στο γ_n με το A_n και λαμβάνουμε έτσι το γ_{n-1} . Επαναλαμβάνοντας την ίδια διαδικασία για το γ_{n-1} θα καταλήξουμε τελικά στο γ_0 , δηλ το αρχικό σύμβολο S της γραμματικής.

Οι αναγωγές σε προτασιακό τύπο δεξιάς παραγωγής «απορροφούν» τα τερματικά σύμβολα από αριστερά προς τα δεξιά!

Έτσι «σκεφτήκαμε» ότι ένας τρόπος parsing με $L \rightarrow R$ ανάγνωση εισόδου είναι ο προσδιορισμός με διαδοχικές αναγωγές της δεξιάς παραγωγής της εισόδου, με αρχικό προτασιακό τύπο τη λέξη εισόδου.

Εκτός ύλης



Αναλυτές LR – ένθετο (1/3)

- Θα κάνουμε μία απλή παρατήρηση κοιτώντας τον τρόπο που δουλεύει ένα s/r αυτόματο για να προσδιορίσει το δέντρο συντακτικής ανάλυσης bottom-up της λέξης εισόδου
- Θα παρατηρούμε τα περιεχόμενα της στοίβας (bottom $\rightarrow$ top) μαζί με τα την είσοδο που έχει απομείνει στην είσοδο (left $\rightarrow$ right) ως έναν προτασιακό τύπο

Εκτός ύλης



Αναλυτές LR – ένθετο (2/3)

1

--

num	*	(	num	+	num	)
-----	---	---	-----	---	-----	---

num (num+num)* *shift*

2

num

*	(	num	+	num	)
---	---	-----	---	-----	---

num (num+num)* *reduce*

3

expr

*	(	num	+	num	)
---	---	-----	---	-----	---

expr (num+num)* *shift*

4

*
expr

(	num	+	num	)
---	-----	---	-----	---

expr (num+num)* *reduce*

5

op
expr

(	num	+	num	)
---	-----	---	-----	---

expr op (num+num)

Εκτός ύλης

6

(
op
expr

num	+	num	)
-----	---	-----	---

expr op (num+num) *shift*

7

num
(
op
expr

+	num	)
---	-----	---

expr op (num+num) *reduce*

8

expr
(
op
expr

+	num	)
---	-----	---

expr op (expr+num) *shift*



Αναλυτές LR – ένθετο (3/3)

9

+	num	)
expr		
(		
op		
expr		

reduce

expr op (expr+num)

10

op	num	)
expr		
(		
op		
expr		

shift

expr op (expr op num)

11

num	)
op	
expr	
(	
op	
expr	

reduce

expr op (expr op num)

12

expr	)
op	
expr	
(	
op	
expr	

reduce

expr op (expr op expr)

13

expr	)
(	
op	
expr	

shift

expr op (expr)

14

)
<u>expr</u>
(
<u>op</u>
<u>expr</u>

reduce

expr op (expr)

15

expr
op
expr

reduce
expr op expr

16

expr

expr

<i>expr</i>	<i>=>rm</i>
<i>expr op expr</i>	<i>=>rm</i>
<i>expr op (expr)</i>	<i>=>rm</i>
<i>expr op (expr op expr)</i>	<i>=>rm</i>
<i>expr op (expr op num)</i>	<i>=>rm</i>
<i>expr op (expr+num)</i>	<i>=>rm</i>
<i>expr op (num+num)</i>	<i>=>rm</i>
<i>expr* (num+num)</i>	<i>=>rm</i>
<i>num* (num+num)</i>	

Εκτός ύλης

Αναλυτές LR (6/14)

Εκτός ύλης

Δεξιός προτασιακός τύπος	Handle	Αναγωγή
$id_1 + id_2 * id_3$	id_1	$E \rightarrow id$
$E + id_2 * id_3$	id_2	$E \rightarrow id$
$E + E * id_3$	id_3	$E \rightarrow id$
$E + E * E$	$E * E$	$E \rightarrow E * E$
$E + E$	$E + E$	$E \rightarrow E + E$
E		

Εφικτά προθέματα – viable prefixes. Στο προηγούμενο παράδειγμα, για κάθε δεξί προτασιακό τύπο, το τμήμα μέχρι και το δεξιότερο handle ονομάζεται εφικτό πρόθεμα. Π.χ. το $E + id_2$ είναι εφικτό πρόθεμα του $E + id_2 * id_3$, ενώ το $E + E * E$ είναι εφικτό πρόθεμα του $E + E * E$.

Προσοχή: για δεδομένο προτασιακό τύπο, εάν η γραμματική δεν είναι διφορούμενη, θα έχουμε ένα και μοναδικό handle. Επίσης, οι ορισμοί αφορούν ολοκληρωμένες (εφικτές) δεξιές παραγωγές.

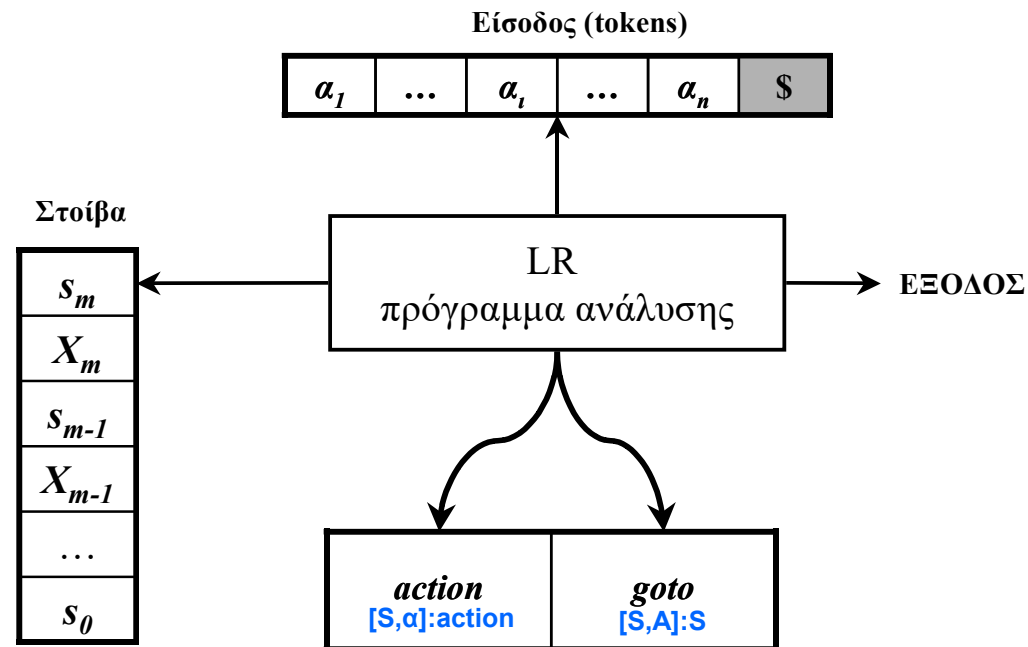
Αναλυτές LR (7/14)

Εκτός ύλης

Συζήτηση. Πρακτικά, το handle ενός δεξιού προτασιακού τύπου σηματοδοτεί το σημείο δεξιότερα του οποίου δεν έχουν γίνει ακόμη αναγωγές, άρα θα υπάρχουν μόνο τερματικά σύμβολα, ενώ αριστερά του (συμπεριλαμβάνοντας το handle) έχουν εφαρμοστεί και αναμένονται επόμενες αναγωγές.

- Έτσι, εάν το τμήμα της λέξης εισόδου που έχουμε διαβάσει μέχρι ενός σημείου έχει γίνει reduced σε εφικτό πρόθεμα, η ανάλυση έχει προχωρήσει χωρίς λάθος.
- Στην πράξη, λόγω του τρόπου λειτουργίας του S/R parser, η ακολουθία των συμβόλων της στοίβας (από κάτω προς τα πάνω) θα πρέπει (για επιτυχή ανάλυση) να είναι πάντα ένα εφικτό πρόθεμα της δεξιάς παραγωγής της λέξης εισόδου.
- Έτσι, ισοδύναμα ορίζουμε τα εφικτά προθέματα ως τις ακολουθίες συμβόλων που μπορούν να εμφανιστούν στη στοίβα ενός S/R parser (στην περίπτωση επιτυχούς ανάλυσης).

Αναλυτές LR (8/14)



Η δομή ενός LR parser φαίνεται στο παραπάνω σχήμα. Ο πίνακας μετάβασης έχει δύο τμήματα (*action* και *goto*). Στη στοίβα αποθηκεύονται ζευγάρια γραμματικών συμβόλων X_i και καταστάσεων s_i . Το πρόγραμμα ανάλυσης είναι το ίδιο για όλους του LR parsers, μόνο ο πίνακας ανάλυσης μεταβάλλεται. Το πρόγραμμα αυτό συμπεριφέρεται ως εξής:



Αναλυτές LR (9/14)

- Έστω s_m στην κορυφή της στοίβα και a_i το επόμενο σύμβολο εισόδου. Τότε το στοιχείο $action[s_m, a_i]$ καθορίζει την επόμενη κίνηση που μπορεί να είναι: (1) *shift* s , για κατάσταση s , (2) *reduce* με κάποιον κανόνα $A \rightarrow \beta$, (3) *accept*, και (4) *error*.
- Η συνάρτηση *goto* αντιστοιχεί κατάσταση και γραμματικό σύμβολο σε κατάσταση. Θεωρητικά, η *goto* είναι η συνάρτηση μετάβασης για ένα DFA που αναγνωρίζει τα εφικτά προθέματα της γραμματικής.
- Ένα *configuration* του LR parser είναι ένα ζευγάρι που αποτελείται από τα περιεχόμενα της στοίβας και το εναπομείναν τμήμα της εισόδου:
$$(s_0 X_1 s_1 X_2 s_2 \dots X_m s_m, a_i a_{i+1} \dots a_n \$)$$
το οποίο ουσιαστικά αναπαριστά τον παρακάτω δεξιό προτασιακό τύπο, όπως ακριβώς θα συνέβαινε και με έναν S/R parser:

$$X_1 X_2 \dots X_m a_i a_{i+1} \dots a_n$$



Αναλυτές LR (10/14)

Ανάλογα λοιπόν με την κίνηση του parser, το νέο configuration του parser για κάθε μία από τις τέσσερις κινήσεις ορίζεται ως εξής:

- Αν $action[s_m, a_i] = shift\ s$, τότε εκτελείται το shift και το νέο configuration είναι $(s_0\ X_1\ s_1\ X_2\ s_2\ \dots\ X_m\ s_m\ a_i s, a_{i+1}\ \dots\ a_n\$)$, δηλ. γίνεται shift στη στοίβα τόσο το σύμβολο εισόδου a_i όσο και η κατάσταση s .
- Αν $action[s_m, a_i] = reduce\ A \rightarrow \beta$, τότε γίνεται κίνηση αναγωγής με νέο configuration $(s_0\ X_1\ s_1\ X_2\ s_2\ \dots\ X_{m-r}\ s_{m-r}\ A s, a_i a_{i+1}\ \dots\ a_n\$)$ όπου r είναι το μήκος σε γραμματικά σύμβολα του β και $goto[s_{m-r}, A] = s$. Ο parser κάνει συνολικά pop $2r$ σύμβολα, αφήνοντας στην κορυφή της στοίβας την κατάσταση s_{m-r} και κάνοντας push το A και το $goto[s_{m-r}, A]$. Ταυτόχρονα, ο parser μπορεί να εκτελέσει και κώδικα που έχει συσχετιστεί με την παραγωγή $A \rightarrow \beta$, καθώς και να εκτυπώσει στην έξοδο την αναγωγή αυτή.
- Αν $action[s_m, a_i] = accept$, τελείωσε επιτυχώς η ανάλυση.
- Αν $action[s_m, a_i] = error$, εντοπίστηκε λάθος και καλείται η συνάρτηση αντιμετώπισης λάθους.



Αναλυτές LR (11/14)

Ο αλγόριθμος για LR parsing

```
i=0;
while true do {
    s = top(), α = input[i];
    if action[s, α] = shift snew then {
        push(α, snew), i = i+1;
    }
    else
    if action[s, α] = reduce A → β then {
        popmany(2*|β|);
        snew = top();
        push(A, goto[snew, A]), output("A → β");
    }
    else if action[s, α] = accept then return;
    else error();
}
```



Αναλυτές LR (12/14)

$expr \rightarrow expr + term$ (1)
 $expr \rightarrow term$ (2)
 $term \rightarrow term * prim$ (3)
 $term \rightarrow prim$ (4)
 $prim \rightarrow (expr)$ (5)
 $prim \rightarrow num$ (6)

Παράδειγμα ενός LR parsing table για την
διπλανή γραμματική απλών αριθμητικών
εκφράσεων

[state, terminal] → S/R/A/E

[state, symbol] → state

STATE	action						goto		
	num	+	*	(	)	\$	expr	term	prim
0	s5			s4			1	2	3
1		s6				accept			
2		r2	s7		r2	r2			
3		r4	r4		r4	r4			
4	s5			s4			8	2	3
5		r6	r6		r6	r6			
6	s5			s4				9	3
7	s5			s4					10
8		s6			s11				
9		r1	s7		r1	r1			
10		r3	r3		r3	r3			
11		r5	r5		r5	r5			

s_i = shift state i , r_i = reduce με τον κανόνα (i)



Αναλυτές LR (13/14)

ΣΤΟΙΒΑ		ΕΙΣΟΔΟΣ	ΚΙΝΗΣΗ
(1)	0	num * num + num\$	shift
(2)	0 num 5	* num + num\$	reduce <i>prim</i> → num
(3)	0 <i>prim</i> 3	* num + num\$	reduce <i>term</i> → <i>prim</i>
(4)	0 <i>term</i> 2	* num + num\$	shift
(5)	0 <i>term</i> 2 * 7	num + num\$	shift
(6)	0 <i>term</i> 2 * 7 num 5	+ num\$	reduce <i>prim</i> → num
(7)	0 <i>term</i> 2 * 7 <i>prim</i> 10	+ num\$	reduce <i>term</i> → <i>term</i> * <i>prim</i>
(8)	0 <i>term</i> 2	+ num\$	reduce <i>expr</i> → <i>term</i>
(9)	0 <i>expr</i> 1	+ num\$	shift
(10)	0 <i>expr</i> 1 + 6	num\$	shift
(11)	0 <i>expr</i> 1 + 6 num 5	\$	reduce <i>prim</i> → num
(12)	0 <i>expr</i> 1 + 6 <i>prim</i> 3	\$	reduce <i>term</i> → <i>prim</i>
(13)	0 <i>expr</i> 1 + 6 <i>term</i> 9	\$	reduce <i>expr</i> → <i>expr</i> + <i>term</i>
(14)	0 <i>expr</i> 1	\$	accept

Παράδειγμα για την είσοδο **num * num + num**

Αναλυτές LR (14/14)

- **LR γραμματικές.** Μία γραμματική για την οποία είναι εφικτή η κατασκευή ενός LR parsing table λέγεται LR γραμματική. Υπάρχουν CF γραμματικές που δεν είναι LR αλλά αυτές μπορούν να αποφευχθούν για κλασικές προγραμματιστικές δομές.
- Ανάλογα με τον αριθμό k των Look-ahead συμβόλων εισόδου η γραμματική ορίζεται ως $LR(k)$. Πρακτικά μας ενδιαφέρουν μόνο οι περιπτώσεις $LR(0)$ και $LR(1)$. Εν γένει για κάθε $k \geq 0$ ισχύει:
 - $LL(k) \subset LR(k)$
 - $LL(k+1) \not\subset LR(k)$ και $LR(k) \not\subset LL(k+1)$

